

**Instituto Tecnológico José Mario Molina Pasquel y
Henríquez**

Unidad Académica Zapotlanejo.

Carrera: Ingeniería Informática (semestre 4)

Materia: Administración y Organización de datos

Actividad: Manual de practicas

Autor: Cristian Rodriguez Rodriguez

Parcial: III

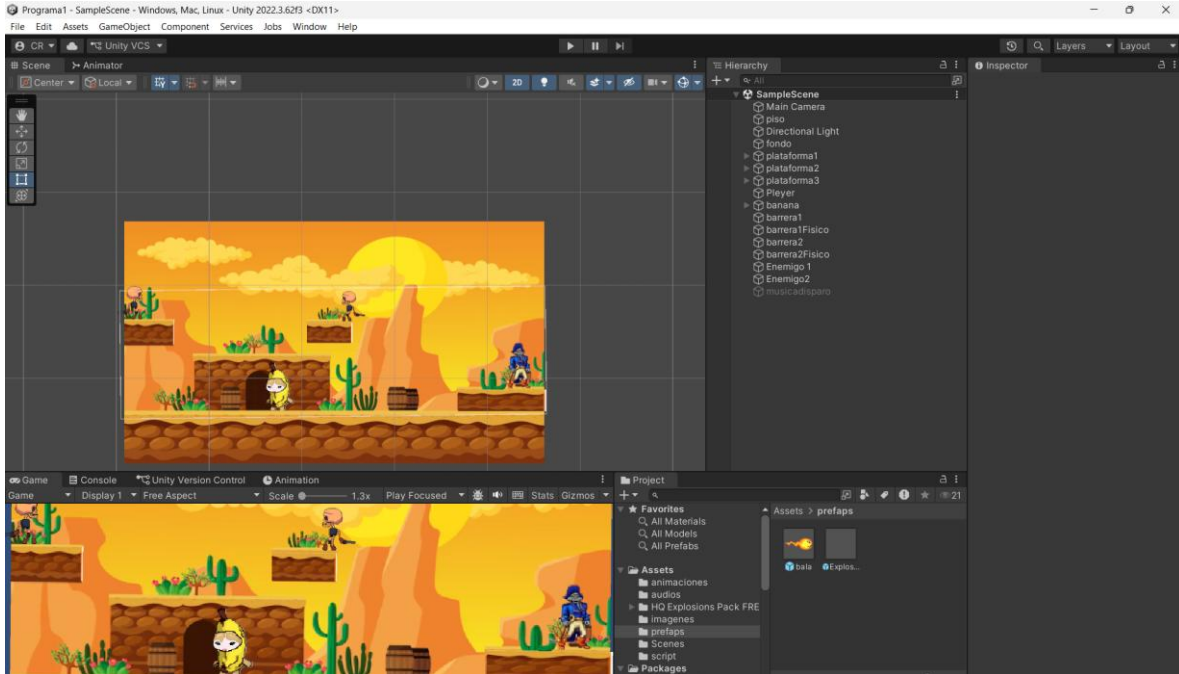
Zapotlanejo, Jalisco, 06 de Mayo de 2026

Índice

Unidad 1	3
Practica 1	3
Unidad 2	7
Practica 2.1	7
Practica 2.2	9
Practica 2.3	14
Unidad 3	20
Practica 1 (Juego de Pelotita):	20
Practica 2 (Preexamen)	28
Unidad 4	33
Practica 1 (Ventanas)	33
Unidad 5	41
Practica 1 (Videojuego de Naves Espaciales)	41

Unidad 1

Practica 1



```
1  using System.Collections;
2  using System.Collections.Generic;
3  using Unity.Burst.Intrinsics;
4  using UnityEngine;
5
6  Script de Unity (1 referencia de recurso) | 0 referencias
7  public class ataque : MonoBehaviour
8  {
9      public float distancia = 30.0f;
10     Vector2 posicion;
11     public GameObject explosion;
12     //public AudioSource audioExplosion;
13
14     // Start is called before the first frame update
15     Mensaje de Unity | 0 referencias
16     void Start()
17     {
18         posicion = transform.position; // la posicion de donde se creo la bala
19         //audioExplosion.Stop();
20     }
21
22     // Update is called once per frame
23     Mensaje de Unity | 0 referencias
24     void Update()
25     {
26         float inicio = Vector2.Distance(posicion, transform.position);
27         inicio = Mathf.Abs(inicio);
28         if (inicio > distancia)
29         {
30             Destroy(gameObject);
31         }
32     }
33 }
```

```

30 }
31 //public void disparar()
32 //{
33 //    // Este código es para invertir la rotación de una imagen "poder"
34 //    Vector2 direccion = mirarHacia ? Vector2.right : Vector2.left; // si esta mirando igual que mi variable entonces
35 //    float angulo = mirarHacia ? 90f : 270f;
36 //    GameObject clon = Instantiate(poder, arma.position, Quaternion.Euler(0, 0, angulo));
37 //    GameObject clon = Instantiate(poder, arma.position, arma.rotation);
38 //    Rigidbody2D cl = clon.GetComponent<Rigidbody2D>(); // se obtienen todos los componentes del rigid body
39 //    cl.velocity = (arma.right * velocidad);
40 //    cl.velocity = direccion * velocidad;
41 //}
42 // Mensaje de Unity | 0 referencias
43 private void OnTriggerEnter2D(Collider2D collision)
44 {
45     if (collision.gameObject.tag == "piso")
46     {
47         Destroy(gameObject); // destruyo el objeto de mi bala si toco el piso
48     }
49     if (collision.gameObject.tag == "enemigo")
50     {
51         //audioExplosion.Play();
52         Destroy(gameObject); // destruyo el objeto de mi bala si le dio a un enemigo
53         GetComponent<Renderer>().enabled = false;
54
55         Destroy(collision.gameObject); // destruyo el objeto del enemigo cuando le dio mi bala
56         GameObject cloneExplosion = Instantiate(explosion, transform.position, transform.rotation); // inicializo la animación de mi explosión
57         Destroy(cloneExplosion, 1.5f); // destruyo el objeto de mi explosión
58     }
59 }
60

```

Ilustración 1 Código del script "ataque"

```

1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  // Script de Unity (1 referencia de recurso) | 0 referencias
6  public class Camara : MonoBehaviour
7  {
8      public AudioSource musica;
9      // Start is called before the first frame update
10     //public Transform personaje; // obtiene la coordenada del personaje
11     //Vector3 posicion;
12     //public float velocidadCamara = 10.0f;
13     // Mensaje de Unity | 0 referencias
14     void Start()
15     {
16         //posicion = transform.position - personaje.position; // busca la posición del personaje, y resta la posición del personaje para saber donde está
17         musica.Play();
18     }
19
20     // Update is called once per frame
21     // Mensaje de Unity | 0 referencias
22     void Update()
23     {
24         //Vector3 nuevaPosicion = personaje.position + posicion;
25         //transform.position = Vector3.Lerp(transform.position, nuevaPosicion, velocidadCamara * Time.deltaTime); // movemos la cámara a la posición del pe
26     }
27 }
28

```

Ilustración 2 Código del script "camara"


```

1  using System.Collections;
2  using System.Collections.Generic;
3  using Unity.Burst.Intrinsics;
4  using UnityEngine;
5  // revisa que no te agregre librerias de mas, en ocaciones genera errores
6
7  @ Script de Unity (1 referencia de recurso) | 0 referencias
8  public class mover : MonoBehaviour
9  {
10     public float velocidad = 10.0f; // variable publica se puede acceder desde unity
11     Rigidbody2D cuerpoBanana; // se crea el objeto de banana
12     float coordX = 0.0f;
13     float coordY = 0.0f;
14     public bool tocar_piso = false; // por defecto comieza elebado , sin tocar el piso
15     Animator anime;
16     bool verDerecha = true; // porque la silueta de mi personaje siempre esta hacia la derecha
17
18     public GameObject poder;
19     public Transform arma;
20     public float velocidadBala = 30.0f;
21     bool mirarHacia = true; // indica que el persoaje esta mirndo hacia la derecha
22
23     @ Mensaje de Unity | 0 referencias
24     void Start()
25     {
26         cuerpoBanana = GetComponent<Rigidbody2D>(); // digo que el cuerpo del banana es igual al objeto que tiene este script
27         anime = GetComponent<Animator>(); // digo que la animacion es igual al componente de animacion que esta asociado a banana
28     }
29     // Update is called once per frame
30     @ Mensaje de Unity | 0 referencias
31     void Update()
32     {
33         coordX = Input.GetAxis("Horizontal"); // obtengo el valor de la coordenada a la cual se esta dirigiendo banana
34         if (coordX != 0) // si se esta moviendo
35         {
36             anime.SetBool("correr",true); // activo la animacion de correr
37             if (coordX > 0 && !verDerecha || coordX < 0 && verDerecha) // si se mueve en una orientacion diferente invierto la orientacion de banana
38             {
39                 GirarPersonaje(); // invierto orientacion de banana
40             }
41         }
42         else
43         {
44             anime.SetBool("correr",false); // si no se detecta cambios en la coordenada indico que no se esta moviendo
45         }
46
47         if (Input.GetButtonDown("Fire1")) // si se preciona la entrada pre establecida como fire 1 se activa la animacion de disparar
48         {
49             print("disparas");
50             anime.SetTrigger("disparar");
51         }
52         // coordY = Input.GetAxis("Vertical");
53
54         //Vector3 mover = new Vector3(coordX, coordY, 0);
55         //transform.position += mover * velocidad * Time.deltaTime;
56
57     }
58
59     1 referencia
60     private void GirarPersonaje()
61     {
62     }
63 
```

```

64         verDerecha = !verDerecha;
65         Vector3 giro = transform.localScale; // permite girar la imagen creando un espejo del personaje manteniendo su escala u posición
66         giro.x *= -1;
67         transform.localScale = giro;
68     }
69 }
70 // Mensaje de Unity | 0 referencias
71 private void FixedUpdate() // este método se utiliza para trabajar con físicas
72 {
73     cuerpoBanana.velocity = new Vector3(coordX * velocidad * Time.deltaTime, 0, 0); // hacemos que se mueva el banano
74     if (cuerpoBanana.velocity.x < 0)
75     {
76     }
77     if (Input.GetKey(KeyCode.Space) && tocar_piso == true)
78     {
79         print("saltando");
80         cuerpoBanana.AddForce(Vector2.up * fuerza, ForceMode2D.Impulse); // hago que banano salte
81         tocar_piso = false;
82         anime.SetBool("brincar", true); // activo animación de saltar
83     }
84 }
85 // Mensaje de Unity | 0 referencias
86 private void OnCollisionEnter2D(Collision2D collision) // esta función se activa cada que mi objeto tenga una colisión
87 {
88     if (collision.gameObject.CompareTag("piso")) // si colisiona con el piso
89     {
90         print("pisando el piso");
91         tocar_piso = true; // esta tocando el piso
92         anime.SetBool("brincar", false); // desactivamos la animación de saltar
93     }
94 }
95 // 0 referencias
96 public void disparar()
97 {
98     // Este código es para invertir la rotación de una imagen "poder"
99     Vector2 direccion = verDerecha ? Vector2.right : Vector2.left; // si esta mirando igual que mi variable entonces
100     float angulo = verDerecha ? 0f : 180f; // rota la bala hacia donde esta apuntando el personaje
101     GameObject clon = Instantiate(poder, arma.position, Quaternion.Euler(0, 0, angulo));
102     // GameObject clon = Instantiate(poder, arma.position, arma.rotation);
103     Rigidbody2D cl = clon.GetComponent<Rigidbody2D>(); // se obtienen todos los componentes del rigid body
104     // cl.velocity = (arma.right * velocidad);
105     cl.velocity = direccion * velocidadBala;
106 }
107 }
108 }
109 }
110 }
111 }

```

Ilustración 3 Código del Script "mover"

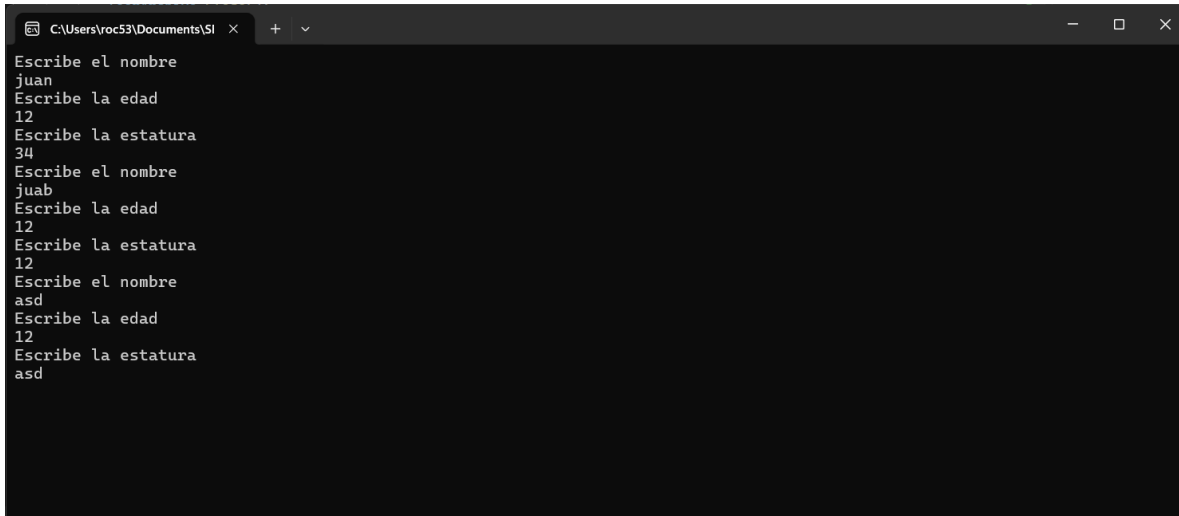
Unidad 2

Practica 2.1

Descripción: En este primer acercamiento con los archivos aprendimos a crear uno y guardar un dato específico en él, al igual de recorrer los registros para posteriormente mostrarlos al usuario.

```
5 static void Main(string[] args)
6 {
7     // Console.WriteLine($"Versión de .NET: {System.Runtime.InteropServices.RuntimeInformation.FrameworkDescription}");
8     string nombreArchivo = "datos.txt";
9     string nombre;
10    int edad;
11    double estatura;
12    StreamWriter archivo = null;
13    StreamReader leerArchivo = null;
14    if (File.Exists(nombreArchivo))
15    {
16        // leemos el archivo
17        leerArchivo = File.OpenText(nombreArchivo);
18        string datos;
19        do
20        {
21            datos = leerArchivo.ReadLine();
22            if (datos != null)
23            {
24                string[] d = datos.Split(" ");
25                Console.WriteLine("el nombre es: " + d[0]); // aqui la "+" si importa, la coma actua difrentes
26
27                Console.WriteLine("La edad es: " + d[1]);
28                Console.WriteLine("La estatura es: " + d[2]);
29            }
30        } while (datos != null); // se repite hasta que el archivo regrese un valor null
31        leerArchivo.Close();
32        return;
33    }
34
35    archivo = File.AppendText(nombreArchivo); // creamos el archivo con el nombre pre definido, esta abierto para escribir
36
37    for (int i = 0; i < 6; i++)
38    {
39        Console.WriteLine("Escribe el nombre");
40        nombre = Console.ReadLine();
41        Console.WriteLine("Escribe la edad");
42        edad = Convert.ToInt32(Console.ReadLine());
43        Console.WriteLine("Escribe la estatura");
44        estatura = Convert.ToDouble(Console.ReadLine());
45
46        // Escribimos los datos en el archivo
47        archivo.WriteLine(nombre + " , " + edad + " , " + estatura);
48    }
49
50    archivo.Close(); // cerramos archivo
51
52
53
54
55
56
57
58
59
```

Ilustración 4 Código Practica 1



```
C:\Users\roc53\Documents\SI x + v
Escribe el nombre
juan
Escribe la edad
12
Escribe la estatura
34
Escribe el nombre
juab
Escribe la edad
12
Escribe la estatura
12
Escribe el nombre
asd
Escribe la edad
12
Escribe la estatura
asd
```

Ilustración 5 Consola de la Practica

Practica 2.2

Descripción: En esta practica se comenzó a trabajar con archivos, nos permitía registrar, mostrar o eliminar un datos, esto mediante la selección de la opción en un menú principal, al finalizar guardamos todo en un archivo y lo mostramos usando el métodos de la clase string

```
4  class Program1
5  {
6      static Validaciones val = new Validaciones();
7      static void Main(string[] args)
8      {
9          // Console.WriteLine($"Versión de .NET: {System.Runtime.InteropServices.RuntimeInformation.FrameworkDescription}");
10         int op = 0;
11         do
12         {
13             op = Menu();
14             switch (op)
15             {
16                 case 1:
17                     Agregar();
18                     break;
19                 case 2:
20                     Console.WriteLine("Eliminar");
21                     EliminarArchivo();
22                     break;
23                 case 3:
24                     Console.WriteLine("Eliminar Dato");
25                     ElimarElementoArchivo();
26                     break;
27                 case 4:
28                     Console.WriteLine("Mostrar Archivo");
29                     Mostrar();
30                     break;
31                 case 5:
32                     Console.WriteLine("Salir");
33                     break;
34             }
35         } while (op != 5);
36     }
37 }
38
39
40
41 1 referencia
42 static void ElimarElementoArchivo()
43 {
44     String nombre;
45     Mostrar();
46     Console.WriteLine("Escribe el nombre a eliminar");
47     nombre = Console.ReadLine();
48     // abrir para buscar
49     if (!File.Exists("datos.txt"))
50     {
51         Console.WriteLine("El archivo no existe");
52         return;
53     }
54     StreamReader leerArchivo = null;
55     leerArchivo = File.OpenText("datos.txt");
56     string datos;
57     do
58     {
59         datos = leerArchivo.ReadLine();
60         if (datos != null)
61         {
62             string[] d = datos.Split(" ", StringSplitOptions.RemoveEmptyEntries);
63             if (nombre.Equals(d[0]))
64             {
65                 Console.WriteLine("Si lo encontro");
66             }
67             else
68             {
69                 Console.WriteLine("Copiando datos");
70                 StreamWriter archivo = File.AppendText("datosAuxiliar.txt");
71                 archivo.WriteLine(datos);
72                 archivo.Close();
73             }
74         }
75     } while (datos != null);
76     leerArchivo.Close();
77 }
```

```

72     }
73 }
74 }
75 }
76 }
77 while (datos != null); // se repite hasta que el archivo regrese un valor null
78 leerArchivo.Close();
79 File.Delete("datos.txt");
80 File.Copy("datosAuxiliar.txt", "datos.txt");
81 if (File.Exists("datosAuxiliar.txt"))
82 {
83     File.Delete("datosAuxiliar.txt");
84 }
85 }
86 }
87 }
88 1 referencia
89 static void EliminarArchivo()
90 {
91     if (File.Exists("datos.txt"))
92     {
93         File.Delete("datos.txt");
94         return;
95     }
96     Console.WriteLine("El archivo no existe");
97 }
98 2 referencias
99 static void Mostrar()
100 {
101     if (!(File.Exists("datos.txt")))
102     {
103         Console.WriteLine("El archivo no existe");
104         return;
105     }
106     StreamReader leerArchivo = null;
107     string datos;
108     do
109     {
110         datos = leerArchivo.ReadLine();
111         if (datos != null)
112         {
113             string[] d = datos.Split(" ", );
114             Console.WriteLine("el nombre es:" + d[0]); // aqui la "+" si importa, la coma actua difrentes
115             Console.WriteLine("La edad es: " + d[1]);
116             Console.WriteLine("La estatura es: " + d[2]);
117         }
118     }
119     while (datos != null); // se repite hasta que el archivo regrese un valor null
120     leerArchivo.Close();
121 }
122 }
123 }
124 1 referencia
125 static void Agregar()
126 {
127     StreamWriter archivo = null;
128     string nombre, edad, estatura;
129     while (true)
130     {
131         Console.WriteLine("Escribe un nombre");
132         nombre = Console.ReadLine();
133         if (val.ValidarNombre(nombre))
134         {
135             break;
136         }
137         //else
138         //{}
139         Console.WriteLine("Error al introducir el nombre");
140     }

```

```

141         //}
142     }
143     // Console.WriteLine("pasaste nombre");
144     while (true)
145     {
146         Console.WriteLine("Escribe la edad");
147         edad = Console.ReadLine();
148         if (val.ValidarEdad(edad))
149         {
150             break;
151         }
152         else
153         {
154             Console.WriteLine("Error al introducir el nombre");
155         }
156     }
157     Console.WriteLine("La edad correcta es: " + edad);
158
159     while (true)
160     {
161         Console.WriteLine("Escribe la estatura");
162         estatura = Console.ReadLine();
163         if (val.ValidarSexo(estatura))
164         {
165             break;
166         }
167         //else
168         //{
169         Console.WriteLine("Error al introducir la estatura correcta");
170         //}
171     }
172     Console.WriteLine("la estatura es correcta es: " + estatura);
173
174     archivo = File.AppendText("datos.txt");
175     archivo.WriteLine(nombre + " , " + edad + " , " + estatura);
176     archivo.Close();
177
178     return;
179
180 }
181
182
183 1 referencia
184 static int Menu()
185 {
186     inicio:
187     int opcion = 0;
188     Console.WriteLine("1.- Agregar elementos al archivo");
189     Console.WriteLine("2.- Eliminar Archivo");
190     Console.WriteLine("3.- Eliminar Datos en el Archivo");
191     Console.WriteLine("4.- Mostrar Contenido del Archivo");
192     Console.WriteLine("5.- Salir");
193     Console.WriteLine("Escribe la Opcion del Menu");
194     String op = Console.ReadLine();
195     if (val.ValidarEdad(op))
196     {
197         opcion = int.Parse(op);
198     }
199     else
200     {
201         Console.WriteLine("Error");
202         goto inicio;
203     }
204     return opcion;
205 }
206

```

Ilustración 6 Código Principal del programa 2

```

8      class Validaciones
9      {
10         1 referencia
11         public bool ValidarNombre(String nombre)
12         {
13             //byte[] ascii = Encoding.ASCII.GetBytes(nombre);
14             //int c = 0;
15             //foreach(byte b in ascii)
16             //{
17                 //if ((b >= 97 && b <= 122) || (b >= 65 && b <= 90) || b == 32)
18                 //{
19                     //c++;
20                 //}
21                 //if (!(b >= 97 && b <= 122) || (b >= 65 && b <= 90) || b == 32))
22                 //{
23                     Console.WriteLine("Nombre erroneo");
24                     return false;
25                 //}
26             //}
27         foreach (char a in nombre)
28         {
29             if (!(a >= 97 && a <= 122) || (a >= 65 && a <= 90) || a == 32))
30             {
31                 Console.WriteLine("Nombre erroneo");
32                 return false;
33             }
34         }
35         //if (c == nombre.Length)
36         //{
37             Console.WriteLine("nombre Correcto");
38             return true;
39         //}
40
41         Console.WriteLine("Nombre Correcto");
42         return true;
43     }
44
45     2 referencias
46     public bool ValidarEdad(String edad)
47     {
48         bool respuesta = (Regex.IsMatch(edad, @"^[0-9]+$")); // revisa que el dato ingresado este siempre dentro de este rango
49         return respuesta;
50     }
51
52     1 referencia
53     public bool ValidarSexo(String esta)
54     {
55         bool r = false;
56         try
57         {
58             float estatura = float.Parse(esta);
59             r = true;
60         }
61         catch (Exception e)
62         {
63             r = false;
64         }
65         return r;
66     }

```

Ilustración 7 Código de validaciones de la Practica 2


```
C:\Users\voc53\Documents\SI x + -
1.- Agregar elementos al archivo
2.- Eliminar Archivo
3.- Eliminar Datos en el Archivo
4.- Mostrar Contenido del Archivo
5.- Salir
Escribe la Opcion del Menu
4
Mostrar Archivo
el nombre es:juan
La edad es: 23
La estatura es: 23
el nombre es:ana
La edad es: 23
La estatura es: 234
el nombre es:pepes
La edad es: 3212
La estatura es: 23
el nombre es:qweqw
La edad es: 12312
La estatura es: 123
1.- Agregar elementos al archivo
2.- Eliminar Archivo
3.- Eliminar Datos en el Archivo
4.- Mostrar Contenido del Archivo
5.- Salir
Escribe la Opcion del Menu
```

Ilustración 8 Aplicacion de consola de la practica

Practica 2.3

Descripción: En esta práctica realizamos un sistema de registro, este solicitaba datos como nombre, teléfono, etc. La principal peculiaridad es que para realizar las validaciones aprendimos a crear nuestras expresiones regulares, un método para validar de forma mas simple y eficiente.

```

24 class Program()
25 {
26     static Validaciones val = new Validaciones();
27     static void Main(string[] args)
28     {
29         int op = 0;
30         do
31         {
32             op = Menu();
33             Console.WriteLine("Estamos seleccionando la opcion");
34             Console.WriteLine(op);
35             switch (op)
36             {
37                 case 1:
38                     Regresar:
39                     GuardarDatos(RegistrarDatos());
40                     Console.WriteLine("Agregar Otro (S/N)");
41                     string resp = Console.ReadLine();
42                     if (resp == "S")
43                     {
44                         goto Regresar;
45                     }
46                     break;
47                 case 2:
48                     Console.WriteLine("Mostrar");
49                     Mostrar();
50                     break;
51                 case 3:
52                     Console.WriteLine("Salir");
53                     break;
54             }
55         } while (op != 3);
56     }
57 }
58
59 static int Menu()
60 {
61     // int respuesta = 0;
62     Console.WriteLine("Selecciona una opcion del menu");
63     Console.WriteLine("1. Registrar Nuevo Usuario");
64     Console.WriteLine("2. Mostrar ");
65     //Console.WriteLine("2. Mostrar datos");
66     Console.WriteLine("3. Salir");
67     string res = Console.ReadLine();
68
69     if (Regex.IsMatch(res, @"^[1-3]+$"))
70     {
71         return int.Parse(res);
72     }
73     return Menu();
74 }
75
76 static string RegistrarDatos()
77 {
78     string datos = "";
79     string nombre, edad, salario, correo, tel, contraseña;
80     Console.WriteLine("Pedimos datos");
81     while (true)
82     {
83         Console.WriteLine("Escribe el nombre: ");
84         nombre = Console.ReadLine();
85         if (val.ValidarString(nombre))
86         {
87             break;
88         }
89         Console.WriteLine("Error al introducir el nombre");
90     }
91     while (true)
92     {
93     }
94 }

```

```
95
96 Console.WriteLine("Escribe la edad");
97 edad = Console.ReadLine();
98 if ((val.ValidarEdad(edad)))
99 {
100     break;
101 }
102 Console.WriteLine("Error al introducir la edad");
103
104 }
105 while (true)
106 {
107
108     Console.WriteLine("Escribe el salario");
109     salario = Console.ReadLine();
110     if ((val.ValidarSalario(salario)))
111     {
112         break;
113     }
114     Console.WriteLine("Error al introducir el salario");
115 }
116 while (true)
117 {
118
119     Console.WriteLine("Escribe el Correo Electronico");
120     correo = Console.ReadLine();
121     if (val.ValidarCorreo(correo))
122     {
123         break;
124     }
125     Console.WriteLine("Error al introducir el correo");
126 }
127
128 while (true)
129 {
130
131     Console.WriteLine("Escribe el numero de Telefono");
132     tel = Console.ReadLine();
133     if (val.ValidarTelefono(tel))
134     {
135         tel = DarFormatoNumero(tel);
136         break;
137     }
138     Console.WriteLine("Error al introducir el telefono");
139 }
140
141 while (true)
142 {
143
144     Console.WriteLine("Escribe la Contraseña");
145     contraseña = Console.ReadLine();
146     if ((val.ValidarContraseña(contraseña)))
147     {
148         break;
149     }
150     Console.WriteLine("Error al introducir el nombre");
151 }
152
153 }
154
155 string formato = " , ";
156 datos = nombre + formato + edad + formato + salario + formato + correo + formato + tel + formato + contraseña;
157 string[] d = datos.Split(" , ");
158 Console.WriteLine("el nombre es: " + d[0]); // aqui la "+" si importa, la coma actua diferentes
159 Console.WriteLine("La edad es: " + d[1]);
160 Console.WriteLine("El salario es: " + d[2]);
161 Console.WriteLine("El correo es: " + d[3]);
162 Console.WriteLine("El numero de telefono es: " + d[4]);
163 Console.WriteLine("La contraseña es: " + d[5].Length);
164
```

```

169         return datos;
170     }
171
172     1 referencia
173     static void GuardarDatos( string datos)
174     {
175         StreamWriter archivo = null;
176         archivo = File.AppendText("datos.txt");
177         archivo.WriteLine(datos);
178         archivo.Close();
179     }
180
181     1 referencia
182     static void Mostrar()
183     {
184         Console.WriteLine("dentro de mostrar");
185         if (!File.Exists("datos.txt"))
186         {
187             Console.WriteLine("El archivo no existe");
188             return;
189         }
190         StreamReader leerArchivo = null;
191         leerArchivo = File.OpenText("datos.txt");
192         string datos;
193         do
194         {
195             datos = leerArchivo.ReadLine();
196             if (datos != null)
197             {
198                 string[] d = datos.Split(" ");
199                 Console.WriteLine("el nombre es: " + d[0]); // aqui la "+" si importa, la coma actua difrentes
200                 Console.WriteLine("La edad es: " + d[1]);
201                 Console.WriteLine("El salario es: " + d[2]);
202                 Console.WriteLine("El correo es: " + d[3]);
203                 Console.WriteLine("El numero de telefono es: " + d[4]);
204                 Console.WriteLine("La longitud de contraseña es: " + d[5].Length);
205             }
206         } while (datos != null); // se repite hasta que el archivo regrese un valor null
207         leerArchivo.Close();
208     }
209
210     1 referencia
211     static string DarFormatoNumero(string tel)
212     {
213         string datosForm = "(" + tel.Substring(0, 3) + ")" + " " + tel.Substring(3, 3) + "-" + tel.Substring(5,4); // el metodo subestrin te pi
214         return datosForm;
215     }
216
217

```

Ilustración 9 Código Principal de la Práctica 3

```

12 public bool ValidarString(string nombre)
13 {
14     //byte[] ascii = Encoding.ASCII.GetBytes(nombre);
15     //int c = 0;
16     //foreach(byte b in ascii)
17     //{
18         //if ((b >= 97 && b <= 122) || (b >= 65 && b <= 90) || b == 32)
19         //{
20             //c++;
21         //}
22         //if (!(b >= 97 && b <= 122) || (b >= 65 && b <= 90) || b == 32))
23         //{
24             //Console.WriteLine("Nombre erroneo");
25             //return false;
26         //}
27     //}
28
29     //// mi version
30     //foreach (char a in nombre)
31     //{
32         //if (!(a >= 97 && a <= 122) || (a >= 65 && a <= 90) || a == 32))
33         //{
34             //Console.WriteLine("Nombre erroneo");
35             //return false;
36         //}
37     //}
38     //if (c == nombre.Length)
39     //{
40         //Console.WriteLine("nombre Correcto");
41         //return true;
42     //}
43     return Regex.IsMatch(nombre, @"^[a-zA-Z\s]+$");
44     //Console.WriteLine("Nombre Correcto");
45     //return true;
46 }
47

```

```

49 public bool ValidarEdad(string edad)
50 {
51     //bool respuesta = (Regex.IsMatch(edad, @"[0-9]+$")); // revisa que el dato ingresado este siempre dentro de este rango
52     //if (respuesta)
53     //{
54         //int e = int.Parse(edad);
55         //if (e >= 18 && e <= 99)
56         //{
57             //return true;
58         //}
59     //}
60     return Regex.IsMatch(edad, @"^(1[8-9]|[2-9][0-9])$");
61 }
62
63

```

```

64 public bool ValidarSalario(string salario)
65 {
66     //try
67     //{
68         //float sal = float.Parse(salario);
69         //salario = salario + ".0";
70         //string[] d = salario.Split(".");
71         //if (d[1].Length <= 2)
72         //{
73             //return true;
74         //}
75         //return false;
76     //}
77     //catch
78     //{
79         //return false;
80     //}
81     return Regex.IsMatch(salario, @"^d+(\.\d{1,2})?$");
82 }
83
84

```

```

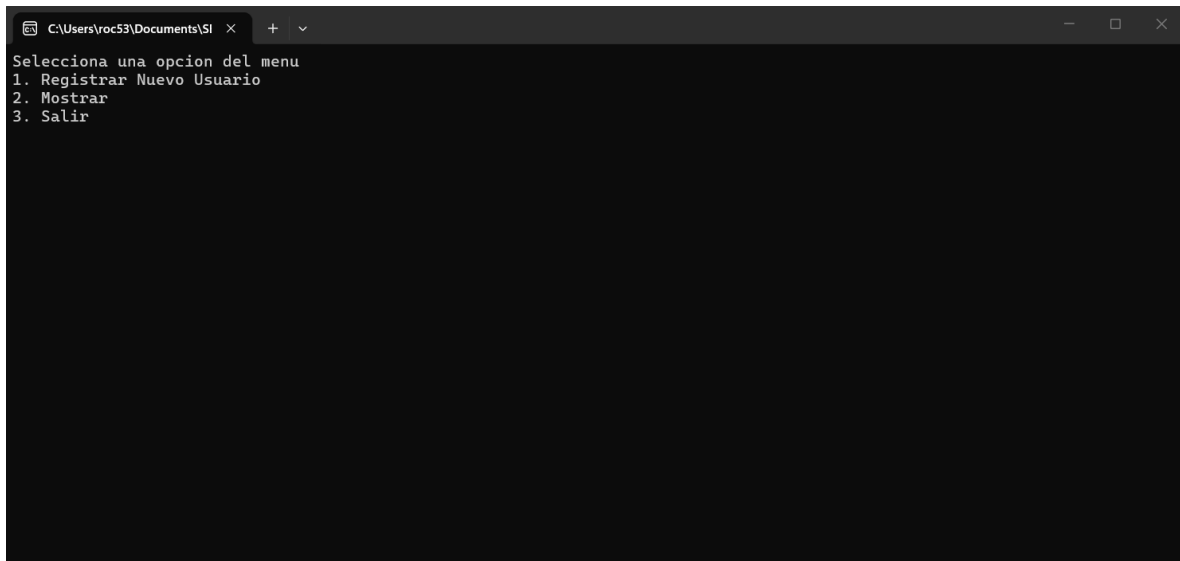
85 public bool ValidarCorreo(string correo)
86 {
87     //try
88     //{
89         string[] subCorreo = correo.Split("@");
90         string[] dominio = subCorreo[1].Split(".");
91         if (dominio[0].Length > 1 && dominio[1].Length > 1 && subCorreo[0].Length > 1)
92         {
93             return true;
94         }
95     }
96     //catch
97     //{
98
99     //}
100     //return false;
101     return Regex.IsMatch(correo, @"^\w-\w+@([\w-]+\w-){2,4}$");
102 }

103 public bool ValidarTelefono(string telefono)
104 {
105     //try
106     //{
107         long tel = long.Parse(telefono);
108         Console.WriteLine(tel);
109         Console.WriteLine(telefono.Length);
110         if (telefono.Length == 10)
111         {
112             return true;
113         }
114     }
115     //catch
116     //{
117         return false;
118     }
119     return Regex.IsMatch(telefono, @"^\d{10}$");
120 }

121 public bool ValidarContraseña(string contrase)
122 {
123     //if (contrase.Length >= 8)
124     //{
125         bool contMay = false;
126         bool contMin = false;
127         bool conyNum = false;
128
129         foreach (char a in contrase)
130         {
131             if (a == 32)
132             {
133                 return false;
134             }
135             if (a >= 'a' && a <= 'z')
136             {
137                 contMin = true;
138             }
139             else if (a >= 'A' && a <= 'Z')
140             {
141                 contMay = true;
142             }
143         }
144         if (contMay && contMin)
145         {
146             return true;
147         }
148     }
149     return Regex.IsMatch(contrase, @"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)\S{8,}$");
150 }

```

Ilustración 10 Validaciones de la practica



```
C:\Users\voc53\Documents\SI x + - x
Selecciona una opcion del menu
1. Registrar Nuevo Usuario
2. Mostrar
3. Salir
```

Ilustración 11 Consola de la App

Unidad 3

Practica 1 (Juego de Pelotita):

Descripción: Se ha realizado un juego que permite guardar el progreso, mediante el uso de archivos, estos datos (nombre y puntos) son mostrados mediante un canva en la pantalla del usuario. También cuenta con dos escenas, que simulan dos niveles diferentes, lo que nos permite aprender mas sobre el manejo de archivos y escenas en Unity.

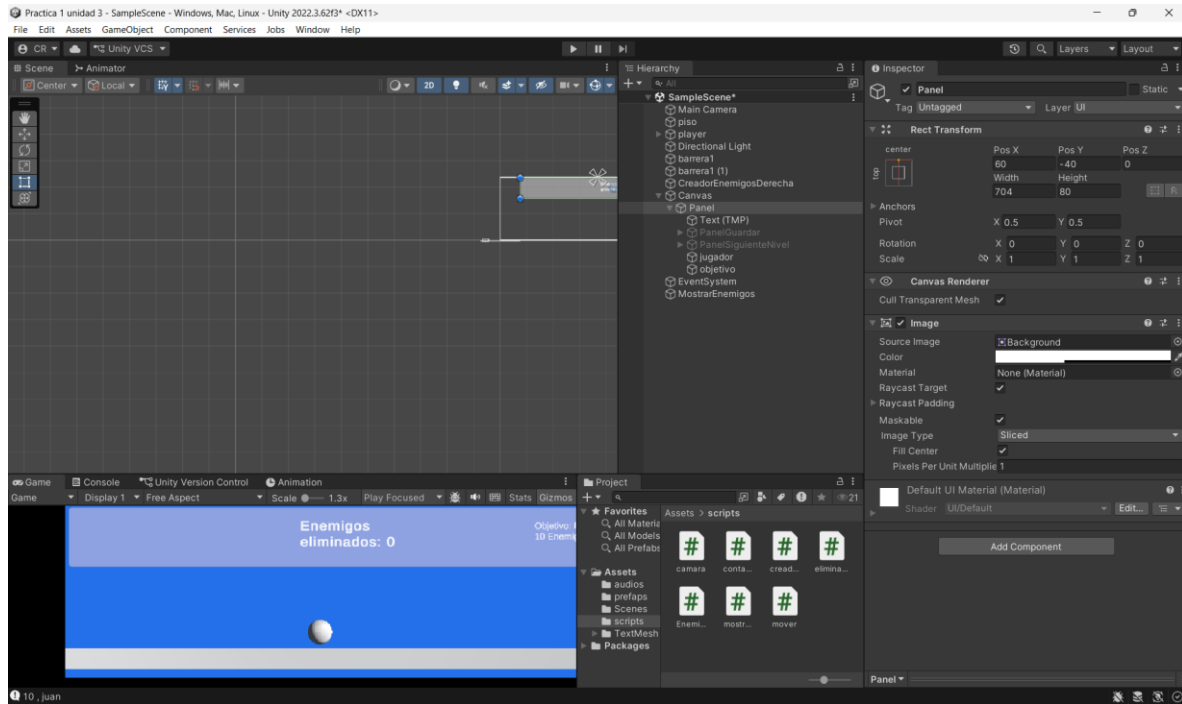


Ilustración 12 Escena 1 del juego

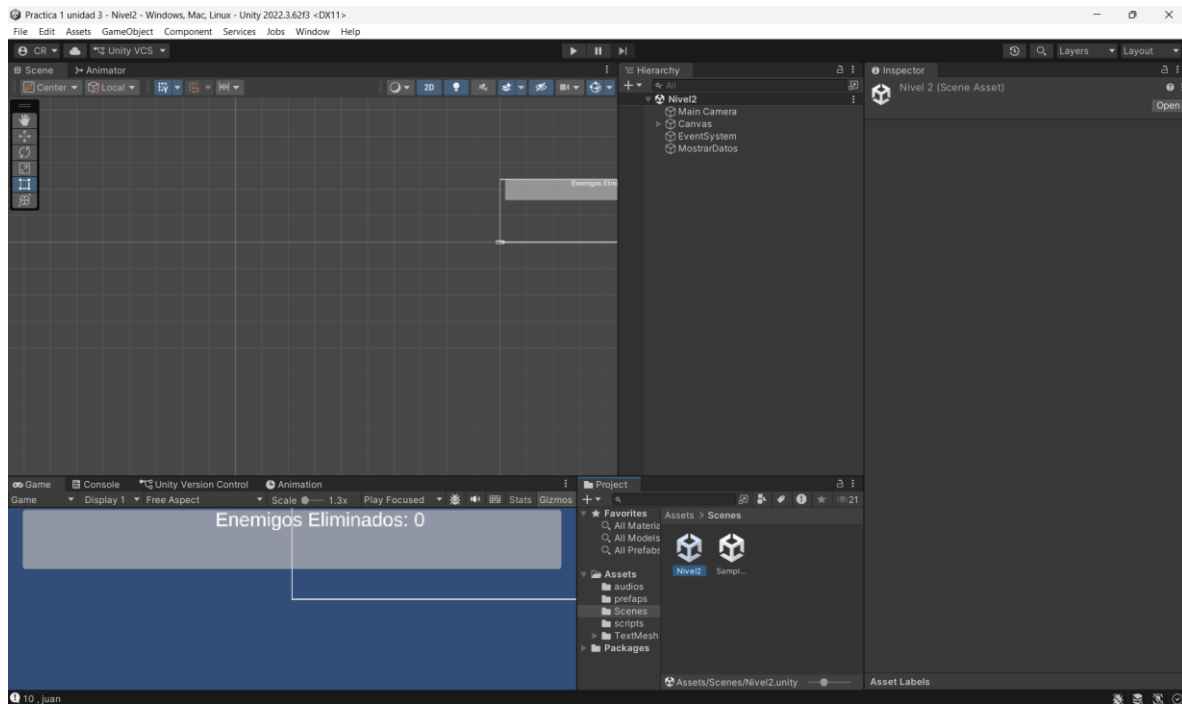


Ilustración 13 escena 2 del juego

```

1  using System.Collections;
2  using System.Collections.Generic;
3  using System.IO;
4  using TMPro;
5  using Unity.VisualScripting.Antlr3.Runtime;
6  using UnityEngine;
7  //using UnityEngine.Windows;
8
9  public class mostrarDatosEscenaAnterior : MonoBehaviour
10 {
11     public TextMeshProUGUI texto;
12     public TextMeshProUGUI player;
13     //public TextMeshProUGUI texto2;
14     //public TextMeshProUGUI mensaje;
15     //public TMP_InputField nombre;
16     // variables comunes
17     public static int contEnemigos = 0;
18     //static StreamWriter archivo = null;
19     static StreamReader leer = null;
20     public static string dato = "0";
21     public static string jugador = "Sin nombre";
22     //public GameObject panel;
23     //public GameObject panelNivel;
24     // Start is called before the first frame update
25     void Start()
26     {
27     }
28
29
30     // Update is called once per frame
31     void Update()
32     {
33         texto.text = "Enemigos Eliminados: " + contEnemigos.ToString();
34     }
35
36     private void Awake()
37     {
38         //int c = 0;
39         if (File.Exists("Puntaje.txt"))
40         {
41             leer = File.OpenText("Puntaje.txt");
42             //print("Leemos el archivo");
43             print(dato);
44             dato = leer.ReadLine();
45             print("Extraemos dato");
46             print(dato);
47             leer.Close();
48             string[] d = dato.Split(" ", " ");
49
50             contEnemigos = int.Parse(d[0]);
51             jugador = d[1];
52         }
53         player.text = jugador;
54     }
55 }
56

```

Ilustración 14 Código de archivo mostrarDatosEscenaAnterior

```

1  //using System.Collections;
2  //using System.Collections.Generic;
3  using System.IO;
4  using TMPro;
5  //using UnityEditor.VersionControl;
6  using UnityEngine;
7  using UnityEngine.SceneManagement;
8
9  Script de Unity (1 referencia de recurso) | 1 referencia
10 public class contadorDeEnemigos : MonoBehaviour
11 {
12     // variables de inferfas grafica
13     public TextMeshProUGUI texto;
14     public TextMeshProUGUI texto2;
15     public TextMeshProUGUI mensaje;
16     public TMP_InputField nombre;
17     public TextMeshProUGUI jugador;
18     string player = "Sin nombre";
19     // variables comunes
20     public static int contEnemigos = 0;
21     static StreamWriter archivo = null;
22     static StreamReader leer = null;
23     public static string dato = "0";
24     public GameObject panel;
25     public GameObject panelNivel;
26
27     // bool est = false;
28
29     // Start is called before the first frame update
30     1 referencia
31     public static void EnemigosEliminados()
32     {
33         contEnemigos++;
34         //CrearArchivo();
35     }
36
37
38     Mensaje de Unity | 0 referencias
39     private void Update()
40     {
41         if (Input.GetKeyDown(KeyCode.G))
42         {
43             print("Guardar Datos");
44             //panel.SetActive(!est);
45             panel.SetActive(!panel.activeSelf); // negamos el estado original de panel
46             Time.timeScale = 0f; // pausa el juego
47             print("pumm");
48             texto2.text = "Datos Guardados: " + contEnemigos.ToString();
49             if (panel.activeSelf)
50             {
51                 mensaje.text = "Mensaje..."; // linea experimental
52             }
53         }
54         else if (panel.activeSelf == false)
55         {
56             Time.timeScale = 1f; // reaunida el juego
57         }
58         texto.text = "Enemigos Eliminados: " + contEnemigos.ToString();
59         if (contEnemigos >= 10)
60         {
61             Time.timeScale = 0;
62             panelNivel.SetActive(true);
63         }
64         jugador.text = player;
65     }
66     Mensaje de Unity | 0 referencias
67     private void Start()
68     {
69     }
70     Mensaje de Unity | 0 referencias

```

```

71 private void Awake()
72 {
73     //int c = 0;
74
75     if (File.Exists("Puntaje.txt"))
76     {
77         Leer = File.OpenText("Puntaje.txt");
78         //print("Leemos el archivo");
79         print(dato);
80         dato = leer.ReadLine();
81         print("Extraemos dato");
82         print(dato);
83         leer.Close();
84         string[] d = dato.Split(" ", " ");
85
86         contEnemigos = int.Parse(d[0]);
87         nombre.text = d[1];
88         player = d[1];
89     }
90 }
91
92 0 referencias
93 public void CrearArchivo()
94 {
95     string n = nombre.text;
96     if (n.Length == 0)
97     {
98         mensaje.text = "No tiene nombre";
99         return;
100     }
101     mensaje.text = "Guardado";
102     if (File.Exists("Puntaje.txt"))
103     {
104         File.Delete("Puntaje.txt");
105     }
106     archivo = File.AppendText("Puntaje.txt");
107     archivo.WriteLine(contEnemigos + " , " + nombre.text);
108     archivo.Close();
109     mensaje.text = "";
110     player = n.ToString();
111 }
112
113 0 referencias
114 public void SiguienteEscena()
115 {
116     GuardarAntesDeNivel();
117     SceneManager.LoadScene("Nivel2"); //loadScene("Nivel2");
118 }
119
120 void GuardarAntesDeNivel()
121 {
122     if (File.Exists("Puntaje.txt"))
123     {
124         File.Delete("Puntaje.txt");
125     }
126     archivo = File.AppendText("Puntaje.txt");
127     archivo.WriteLine(contEnemigos + " , " + nombre.text);
128     archivo.Close();
129 }

```

Ilustración 15 Código de archivo contadorDeEnemigos

```

1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  // Script de Unity (1 referencia de recurso) | 0 referencias
6  public class eliminarEnemigos : MonoBehaviour
7  {
8      // Start is called before the first frame update
9      public AudioSource enemigoMuerte;
10     // Mensaje de Unity | 0 referencias
11     void Start()
12     {
13         enemigoMuerte.Play();
14     }
15
16     // Update is called once per frame
17     // Mensaje de Unity | 0 referencias
18     void Update() { enemigoMuerte.Play(); }
19
20     // Mensaje de Unity | 0 referencias
21     private void OnTriggerEnter2D(Collider2D collision)
22     {
23         if (collision.gameObject.tag == "enemigo")
24         {
25             enemigoMuerte.Play();
26             print("eliminar");
27             Destroy(collision.transform.root.gameObject);
28             contadorDeEnemigos.EnemigosEliminados(); // mandamos llamar a la funcion de otro scrip para aumtenar el contador
29         }
30     }

```

Ilustración 16 Código de archivo *eliminarEnemigos*

```

1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4  using UnityEngine.UIElements;
5  using UnityEngine;
6
7  // Script de Unity (1 referencia de recurso) | 0 referencias
8  public class EnemigoMover : MonoBehaviour
9  {
10     public float velocidad = 50.0f;
11     public float distancia = 30f;
12     Vector2 posicion;
13     // Start is called before the first frame update
14     // Mensaje de Unity | 0 referencias
15     void Start()
16     {
17         posicion = transform.position;
18     }
19
20     // Update is called once per frame
21     // Mensaje de Unity | 0 referencias
22     void Update()
23     {
24         transform.Translate(Vector3.left * velocidad * Time.deltaTime);
25         float inicio = Vector2.Distance(posicion, transform.position);
26         inicio = Mathf.Abs(inicio);
27         if (inicio > distancia)
28         {
29             Destroy(gameObject);
30         }
31     }

```

Ilustración 17 Código de archivo *EnemigoMover*

```

1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 // Script de Unity (1 referencia de recurso) | 0 referencias
6 public class creadorEnemigos : MonoBehaviour
7 {
8     // creador enemigos derechos
9     public GameObject macabron;
10    public float tiempoParaGenerar = 3.0f;
11    public Transform[] posicionD;
12    // Start is called before the first frame update
13    // Mensaje de Unity | 0 referencias
14    void Start()
15    {
16        StartCoroutine(Generar());
17    }
18
19    // Update is called once per frame
20    // Mensaje de Unity | 0 referencias
21    void Update()
22    {
23    }
24
25    1 referencia
26    IEnumerator Generar()
27    {
28        while (true)
29        {
30            Ceneimigos();
31            yield return new WaitForSeconds(tiempoParaGenerar);
32        }
33    }
34
35    1 referencia
36    void Ceneimigos()
37    {
38        if (macabron == null || posicionD.Length == 0)
39        {
40            return;
41        }
42        Transform punto = posicionD[Random.Range(0, posicionD.Length)];
43        Instantiate(macabron, punto.position, punto.rotation);
44    }
45 }

```

Ilustración 18 Código de archivo creadorEnemigos

```

1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 // Script de Unity (1 referencia de recurso) | 0 referencias
6 public class camara : MonoBehaviour
7 {
8     public AudioSource musica;
9     // Start is called before the first frame update
10    public Transform personaje; // obtiene la coordenada del personaje
11    Vector3 posicion;
12    Vector3 fueraCamara;
13    float coorY;
14    public float minCordX;
15    public float maxCordX;
16    public float velocidadCamara = 10.0f;
17    // Mensaje de Unity | 0 referencias
18    void Start()
19    {
20        //posicion = transform.position - personaje.position; // busca la posicion del personaje, y resta la posicion del personaje para saber donde esta
21        musica.Play();
22        fueraCamara = transform.position - personaje.position;
23        coorY = transform.position.y;
24    }
25
26    // Update is called once per frame
27    //void Update()
28    //
29    //
30    // Vector3 nuevaPosicion = personaje.position + posicion;
31    // transform.position = Vector3.Lerp(transform.position, nuevaPosicion, velocidadCamara * Time.deltaTime); // movemos la camara a la posicion del personaje
32    //
33    // Mensaje de Unity | 0 referencias
34    private void LateUpdate()
35    {
36        float objetivo = personaje.position.x + fueraCamara.x;
37        objetivo = Mathf.Clamp(objetivo, minCordX, maxCordX);
38        Vector3 objetivoX = new Vector3(objetivo, coorY, transform.position.z); // coordenadas de mi objetivo
39        transform.position = Vector3.Lerp(transform.position, objetivoX, velocidadCamara * Time.deltaTime); // mandamos a la camara a vijilar
40    }
41 }

```

Ilustración 19 Código de archivo camara

```

1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4  using UnityEngine;
5  // revisa que no te agregue librerias de mas, en ocasiones genera errores
6
7  // Script de Unity (1 referencia de recurso) 0 referencias
8  public class mover : MonoBehaviour
9  {
10     public float velocidad = 10.0f; // variable publica se puede acceder desde unity
11     Rigidbody2D cuerpoBanana; // se crea el objeto de banana
12     float coordX = 0.0f;
13     //float coordY = 0.0f;
14     public bool tocar_piso = false; // por defecto conieza elevado , sin tocar el piso
15     public float fuerza = 10.0f; // fuerza del brico
16     //Animator anime;
17     //bool verDerecha = true; // porque la silueta de mi personaje siempre esta hacia la derecha
18
19     //public GameObject poder;
20     //public Transform arma;
21     public float velocidadBala = 30.0f;
22     //bool mirarHacia = true; // indica que el personaje esta mirndo hacia la derecha
23
24     // Mensaje de Unity (0 referencias)
25     void Start()
26     {
27         cuerpoBanana = GetComponent<Rigidbody2D>(); // digo que el cuerpo del banana es igual al objeto que tiene este script
28         //anime = GetComponent<Animator>(); // digo que la animacion es igual al componente de animacion que esta asociado a banana
29     }
30     // Update is called once per frame
31     // Mensaje de Unity (0 referencias)
32     void Update()
33     {
34
35     void Update()
36     {
37         coordX = Input.GetAxis("Horizontal"); // obtengo el valor de la coordenada a la cual se esta dirigiendo banana
38         //if (coordX != 0) // si se esta moviendo
39
40     private void FixedUpdate() // este metodo se utiliza para trabajar con fisicas
41     {
42         //cuerpoBanana.velocity = new Vector3(coordX * velocidad * Time.deltaTime, 0, 0); // hacemos que se mueva el banana
43         cuerpoBanana.velocity = new Vector2(coordX * velocidad, cuerpoBanana.velocity.y);
44         //if (cuerpoBanana.velocity.x < 0)
45         //{
46
47         //}
48         if (Input.GetKey(KeyCode.Space) && tocar_piso == true)
49         {
50             //print("saltando");
51             cuerpoBanana.AddForce(Vector2.up * fuerza, ForceMode2D.Impulse); // hago que banana salte
52             tocar_piso = false;
53             //anime.SetBool("brincar", true); // activo animacion de saltar
54
55         }
56         if (cuerpoBanana.velocity.y < 0) // si esta elevado eb y, entonces
57         {
58             cuerpoBanana.gravityScale = 30;
59         }
60         else
61         {
62             cuerpoBanana.gravityScale = 8;
63         }
64     }
65
66     // Mensaje de Unity (0 referencias)
67     private void OnCollisionEnter2D(Collision2D collision) // esta funcion se activa cada que mi objeto tenga una colicion por box colalider
68     {
69         if (collision.gameObject.CompareTag("piso")) // si coliciona con el piso
70         {
71             //print("pisando el piso");
72             tocar_piso = true; // esta tocando el piso
73             //anime.SetBool("brincar", false); // desactivamos la animacion de saltar
74         }
75     }
76

```

Ilustración 20 Código de archivo mover

Practica 2 (Preexamen)

Descripción: Con la finalidad de prepararnos para el examen se nos asignó la misión de recrear el juego de la ranita, este debe permitir a un personaje (ranita) pasar por un escenario mientras a su alrededor se generan frutas (que suman puntos) y enemigos (que la regresan al principio).

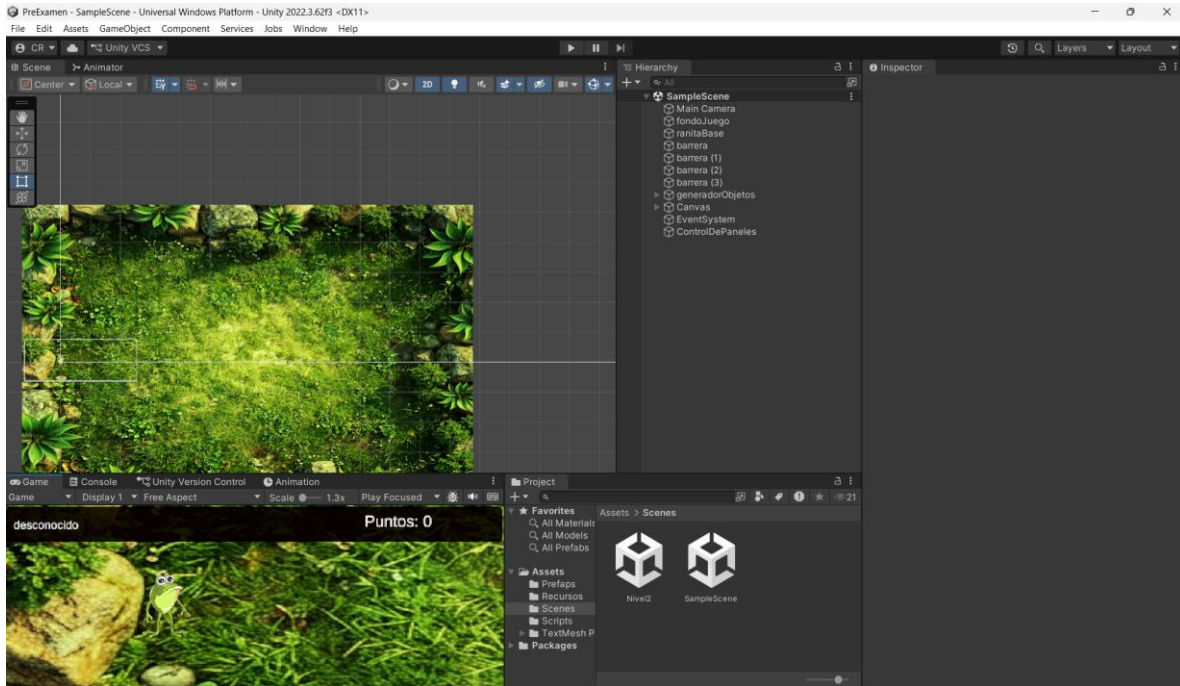


Ilustración 21 escena 1

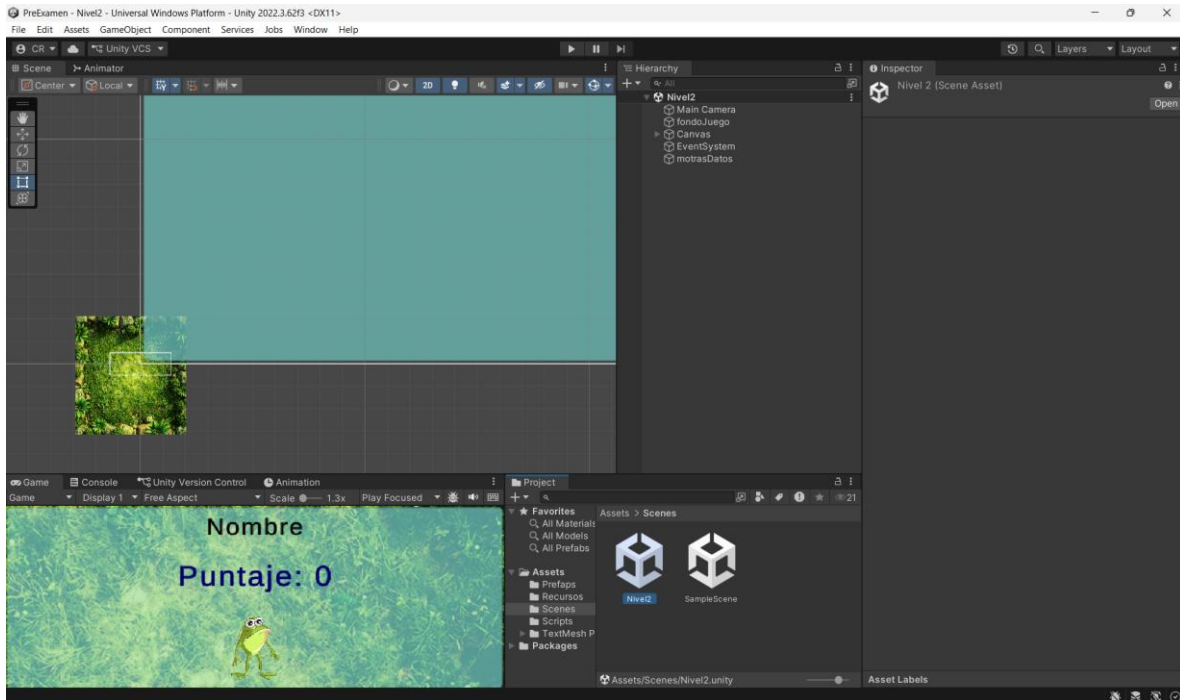


Ilustración 22 escena 2

```

1  //using System.Collections;
2  //using System.Collections.Generic;
3  using System.IO;
4  using TMPro;
5  //using Unity.VisualScripting.Antlr3.Runtime;
6  using UnityEngine;
7  //using UnityEngine.Windows;
8
9  // Script de Unity (1 referencia de recurso) | 0 referencias
10 public class mostrarDatosEscenaAnterior : MonoBehaviour
11 {
12     public TextMeshProUGUI texto;
13     public TextMeshProUGUI player;
14     //public TextMeshProUGUI texto2;
15     //public TextMeshProUGUI mensaje;
16     //public TMP_InputField nombre;
17     // variables comunes
18     public static int contEnemigos = 0;
19     //static StreamWriter archivo = null;
20     static StreamReader leer = null;
21     public static string dato = "0";
22     public static string jugador = "Sin nombre";
23     //public GameObject panel;
24     //public GameObject panelNivel;
25     // Start is called before the first frame update
26     // Mensaje de Unity | 0 referencias
27     void Start()
28     {
29     }
30
31     // Update is called once per frame
32     // Mensaje de Unity | 0 referencias
33     void Update()
34     {
35         texto.text = "Puntaje: " + contEnemigos.ToString();
36     }
37
38     // Mensaje de Unity | 0 referencias
39     private void Awake()
40     {
41         //int c = 0;
42
43         //if (File.Exists("pajaro.txt"))
44         {
45             leer = File.OpenText("pajaro.txt");
46             //print("Leemos el archivo");
47             print(dato);
48             dato = leer.ReadLine();
49             print("Extraemos dato");
50             print(dato);
51             leer.Close();
52             string[] d = dato.Split(" ");
53
54             contEnemigos = int.Parse(d[0]);
55             jugador = d[1];
56         }
57     }
58
59     player.text = jugador;
60 }

```

Ilustración 23 Código de archivo mostrarDatosEscenaAnterior

```

1  //using System.Collections;
2  //using System.Collections.Generic;
3  using System.IO;
4  using TMPro;
5  //using UnityEditor.VersionControl;
6  using UnityEngine;
7  using UnityEngine.SceneManagement;
8
9  // Script de Unity (1 referencia de recurso) | 1 referencia
10 public class contadorDeEnemigos : MonoBehaviour
11 {
12     // variables de interfaz grafica
13     public TextMeshProUGUI text01Panel;
14     //public TextMeshProUGUI text02;
15     //public TextMeshProUGUI mensaje;
16     public TMP_InputField nombre;
17     public TextMeshProUGUI jugador;
18     public TextMeshProUGUI puntosP2;
19     string player = "Desconocido";
20     // variables comunes
21     public static int contEnemigos = 0;
22     static StreamWriter archivo = null;
23     static StreamReader leer = null;
24     public static string dato = "0";
25     // declara los pantes que vas a abrir
26     //public GameObject panel;
27     public GameObject panelNivel;
28
29     // bool est = false;
30
31     // Start is called before the first frame update
32     1 referencia
33     public static void EnemigosEliminados()
34     {
35         contEnemigos++;
36         //CrearArchivo();
37     }
38
39
40
41     private void Update()
42     {
43         text01Panel.text = "Puntaje: " + contEnemigos.ToString();
44         if (contEnemigos >= 10)
45         {
46             Time.timeScale = 0;
47             panelNivel.SetActive(true);
48             puntosP2.text = "Puntaje: " + contEnemigos.ToString();
49         }
50         jugador.text = player;
51     }
52     // Mensaje de Unity | 0 referencias
53     private void Start()
54     {
55     }
56
57     // Mensaje de Unity | 0 referencias
58     private void Awake()
59     {
60         //int c = 0;
61
62         if (File.Exists("Puntaje.txt"))
63         {
64             leer = File.OpenText("Puntaje.txt");
65             //print("Leemos el archivo");
66             print(dato);
67             dato = leer.ReadLine();
68             print("Extraemos dato");
69             print(dato);
70             leer.Close();
71             string[] d = dato.Split(" , ");
72
73             contEnemigos = int.Parse(d[0]);
74             nombre.text = d[1];
75             player = d[1];
76         }
77     }
78
79     public void CrearArchivo()
80     {
81         string n = nombre.text;
82         if (n.Length == 0)
83         {
84             //mensaje.text = "No tiene nombre";
85             return;
86         }
87         //mensaje.text = "Guardado";
88         if (File.Exists("Puntaje.txt"))
89         {
90             File.Delete("Puntaje.txt");
91         }
92         archivo = File.AppendText("Puntaje.txt");
93         archivo.WriteLine(contEnemigos + " , " + nombre.text);
94         archivo.Close();
95         //mensaje.text = "";
96         player = n.ToString();
97     }
98
99     // 0 referencias
100     public void SiguienteEscena()
101     {
102         GuardarAntesDeNivel();
103         SceneManager.LoadScene("Nivel2"); //loadScene("Nivel2");
104     }
105     // 1 referencia
106     void GuardarAntesDeNivel()
107     {
108         if (File.Exists("Puntaje.txt"))
109         {
110             File.Delete("Puntaje.txt");
111         }
112         archivo = File.AppendText("Puntaje.txt");
113         archivo.WriteLine(contEnemigos + " , " + nombre.text);
114         archivo.Close();
115     }

```

Ilustración 24 Código de archivo contadorDeEnemigos

```

1 //using System.Collections;
2 //using System.Collections.Generic;
3 using UnityEngine;
4
5 public class eliminarAltocarBorde : MonoBehaviour
6 {
7     public float velocidad = 50.0f;
8     public float distancia = 30f;
9     Vector2 posicion;
10    // Start is called before the first frame update
11    // Mensaje de Unity | 0 referencias
12    void Start()
13    {
14        posicion = transform.position;
15    }
16
17    // Update is called once per frame
18    // Mensaje de Unity | 0 referencias
19    void Update()
20    {
21        //transform.Translate(Vector3.left * velocidad * Time.deltaTime); // indica que se mueva hacia la izquierda "left" (-1,0,0)
22        //transform.Translate(Vector3.down * velocidad * Time.deltaTime); // indica que se mueva hacia abajo "down" (0,-1,0)
23        float inicio = Vector2.Distance(posicion, transform.position); // calcula la distancia del objeto respecto a donde se instancia
24        inicio = Mathf.Abs(inicio); // obtiene el valor absoluto de un dato abs(-5) = 5
25        if (inicio > distancia)
26        {
27            Destroy(gameObject);
28        }
29    }
30
31    // Mensaje de Unity | 0 referencias
32    private void OnTriggerEnter2D(Collider2D collision)
33    {
34        if (collision.gameObject.tag == "limite")
35        {
36            Destroy(gameObject);
37        }
38    }
39
40 }

```

Ilustración 25 Código de archivo eliminarAltocarBorde

```

5 public class GenerarObjeto : MonoBehaviour
6 {
7     // creador enemigos despues
8     public GameObject macabron;
9     public float tiempoParaGenerar = 3.0f;
10    public Transform[] posicion0;
11    // Start is called before the first frame update
12    // Mensaje de Unity | 0 referencias
13    void Start()
14    {
15        StartCoroutine(Generar());
16    }
17
18    // Update is called once per frame
19    // Mensaje de Unity | 0 referencias
20    void Update()
21    {
22    }
23
24    1 referencia
25    IEnumerator Generar()
26    {
27        while (true)
28        {
29            Ceneimigos();
30            yield return new WaitForSeconds(tiempoParaGenerar);
31        }
32    }
33
34    1 referencia
35    void Ceneimigos()
36    {
37        if (macabron == null || posicion0.Length == 0)
38        {
39            return;
40        }
41        Transform punto = posicion0[Random.Range(0, posicion0.Length)];
42        Instantiate(macabron, punto.position, punto.rotation);
43    }
44
45 }

```

Ilustración 26 Código de archivo GenerarObjeto

```

5 public class SeguirObjeto : MonoBehaviour
6 {
7     //public AudioSource musica;
8     // Start is called before the first frame update
9     public Transform personaje; // objeto al cual va a seguir
10    //Vector3 posicion;
11    Vector3 fueraCamara;
12    float coordY;
13    public float minCordX;
14    public float maxCordX;
15    public float minCordY;
16    public float maxCordY;
17    public float velocidadCamara = 10.0f;
18    // Mensaje de Unity | 0 referencias
19    void Start()
20    {
21        //posicion = transform.position - personaje.position; // busca la posicion del personaje, y resta la posicion del personaje para saber donde esta
22        //musica.Play();
23        fueraCamara = transform.position - personaje.position;
24        //coordY = transform.position.y;
25    }
26
27    // Update is called once per frame
28    //void Update()
29    //{
30        // Vector3 nuevaPosicion = personaje.position + posicion;
31        // transform.position = Vector3.Lerp(transform.position, nuevaPosicion, velocidadCamara * Time.deltaTime); // movemos la camara a la posicion del personaje
32    //}
33    // Mensaje de Unity | 0 referencias
34    private void LateUpdate()
35    {
36        float objetivo = personaje.position.x + fueraCamara.x;
37        objetivo = Mathf.Clamp(objetivo, minCordX, maxCordX);
38        float objetivoY = personaje.position.y + fueraCamara.y;
39        objetivoY = Mathf.Clamp(objetivoY, minCordY, maxCordY);
40        Vector3 objetivoX = new Vector3(objetivo, objetivoY, transform.position.z); // coordenadas de mi objetivo
41        transform.position = Vector3.Lerp(transform.position, objetivoX, velocidadCamara * Time.deltaTime); // mandamos a la camara a viajar
42    }
43 }

```

Ilustración 27 Código de archivo SeguirObjeto

```

1 //using System.Collections;
2 //using System.Collections.Generic;
3 using UnityEngine;
4
5 // Script de Unity | 1 referencias de recipien | 0 referencias
6 public class MoverJugador : MonoBehaviour
7 {
8     public float velocidad = 10.0f; // variable publica se puede acceder desde unity
9     Rigidbody2D cuerpoBanana; // se crea el objeto de banana
10    float coordX = 0.0f;
11    float coordY = 0.0f;
12    public bool tocar_piso = false; // por defecto comienza elebado , sin tocar el piso
13    public float fuerza = 10.0f; // fuerza del brico
14    public AudioSource musicaFruta;
15    public AudioSource musicaEnemigo;
16
17    public float velocidadBala = 30.0f;
18
19    // Mensaje de Unity | 0 referencias
20    void Start()
21    {
22        cuerpoBanana = GetComponent<Rigidbody2D>();
23    }
24
25    // Update is called once per frame
26    // Mensaje de Unity | 0 referencias
27    void Update()
28    {
29        coordX = Input.GetAxis("Horizontal");
30        if (coordX < 0.0f) coordX = 0.0f;
31        coordY = Input.GetAxis("Vertical");
32    }
33
34    //private void GirarPersonaje()
35    //{
36        // verDerecha = !verDerecha;
37        // Vector3 giro = transform.localScale; // permite girar la imagen creando un espejo del personaje manteniendo su escala u posicion
38        // giro.x *= -1;
39        // transform.localScale = giro;
40    //}
41
42    //private void FixedUpdate() // este metodo se utiliza para trabajar con fisicas
43    //{
44        //cuerpoBanana.velocity = new Vector3(coordX * velocidad * Time.deltaTime, 0, 0); // hacemos que se mueva el banana
45        //cuerpoBanana.velocity = new Vector2(coordX * velocidad, coordY * velocidad);
46        //if (cuerpoBanana.velocity.x < 0)
47        //{
48            //}
49        //}
50
51    // Mensaje de Unity | 0 referencias
52    private void OnTriggerEnter2D(Collider2D collision)
53    {
54        if (collision.gameObject.tag == "fruta")
55        {
56            musicaFruta.Play();
57            //print("colisiona");
58            Destroy(collision.transform.root.gameObject);
59            contadorDeEnemigos.EnemigosEliminados(); // mandamos llamar a la funcion de otro scrip para auستنar el contador
60        }
61        else if (collision.gameObject.tag == "enemigo")
62        {
63            musicaEnemigo.Play();
64            print("colicion con enemigo");
65            cuerpoBanana.transform.position = new Vector3(0, 0, -1);
66        }
67    }
68 }

```

Ilustración 28 Código de archivo MoverJugador

Unidad 4

Practica 1 (Ventanas)

Descripción: En esta practica se desarrollo un programa que nos permite gestionar nombres de usuarios y contraseñas, mediante el uso archivos “.csv”. Para ello se desarrollaron 2 ventanas, la primera de ellas solicita la introducción de un usuario y contraseña previamente registrados, en caso de no contar con ellos tendrán que guardar un nuevo usuario y contraseña, de lo contrario no podrán acceder al sistema. En la segunda ventana, podemos aprecia todos los usuarios registrados y su respectiva contraseña, adicional a esto nos permite eliminar o modificar algún registro, eso si, siempre y cuando hayamos seleccionado previamente un dato de nuestra tabla.

```
3 namespace practica_1
4 {
5     3 referencias
6     public partial class Practica1 : Form
7     {
8         crud manejadorArchivos = new crud();
9         1 referencia
10        public Practica1()
11        {
12            InitializeComponent();
13        }
14
15        1 referencia
16        private void btnAceptar_Click(object sender, EventArgs e)
17        {
18            string usuario, pasware;
19            usuario = txtUsuario.Text;
20            pasware = txtPasware.Text;
21            if (usuario.Length == 0 || pasware.Length == 0)
22            {
23                MessageBox.Show("Error Faltan Datos", "Error"); // permite colocar texto, y titurlo a la ventan emergente
24                return;
25            }
26            // corroboramos si el archivo existe
27            //if (File.Exists("usuario.csv"))
28            //{
29                MessageBox.Show("El acrivo, si existe");
30            //}
31            //else
32            //{
33                MessageBox.Show("NO hay usuarios Registrados");
34            //}
35            if (!File.Exists("usuario.csv"))
36            {
37                MessageBox.Show("No hay usuarios Registrados");
38                return;
39            }
40            // MessageBox.Show("El archivo, si existe");
41            StreamReader leerArchivo = File.OpenText("usuario.csv");
42            string datoExtraido = "";
43            bool estado = false;
44
45            do
46            {
47            }
```

```

44      datoExtraido = leerArchivo.ReadLine();
45      if (datoExtraido != null)
46      {
47          string[] dartos = datoExtraido.Split(" ");
48          if (usuario.Equals(dartos[0]) && pasware.Equals(dartos[1]))
49          {
50              // MessageBox.Show("El usuario si existe");
51              estado = true;
52              break;
53          }
54      }
55  }
56  }
57
58  while (datoExtraido != null); // se repite mientras el renglon no este bacio
59
60  leerArchivo.Close();
61  txtPasware.Clear();
62  txtUsuario.Clear();
63
64
65  if (!estado)
66  {
67      MessageBox.Show("El usuario no existe", "Error");
68      return;
69  }
70  crud ventCrud = new crud();
71  this.Hide(); // ocultamos la ventana actual
72  ventCrud.ShowDialog(); // madamos a llamar la otra vetana
73
74
75
76
77  1 referencia
78  private void btnGuardar_Click(object sender, EventArgs e)
79  {
80      string usuario, pasware;
81      usuario = txtUsuario.Text;
82      pasware = txtPasware.Text;
83      if (usuario.Length == 0 || pasware.Length == 0)
84      {
85          MessageBox.Show("Error Faltan Datos", "Error"); // permite colocar texto, y titirlo a la ventan emergente
86          return;
87      }
88      if (manejadorArchivos.Coincidencias(usuario))
89      {
90          MessageBox.Show("El nombre de usuario \n ya ha sido registrado");
91          txtUsuario.Clear();
92          txtPasware.Clear();
93          return;
94      }
95      StreamWriter archivo = null;
96      archivo = File.AppendText("usuario.csv"); // cramos el archivo
97      archivo.WriteLine(usuario + " , " + pasware);
98      archivo.Close();
99      MessageBox.Show("El usuario se ha guardado", "Guardado");
100      // borramos las cajas de texto
101      txtUsuario.Clear();
102      txtPasware.Clear();
103
104  1 referencia
105  private void btnSalir_Click(object sender, EventArgs e)
106  {
107      Application.Exit();
108  }
109
110  1 referencia
111  private void txtUsuario_KeyPress(object sender, KeyPressEventArgs e)
112  {
113      //char c = e.KeyChar; // detecta cada caracter que se este precionando en el teclado
114      //MessageBox.Show(c.ToString());
115      if (!(e.KeyChar >= 97 && e.KeyChar <= 122
116          || e.KeyChar == 32 || e.KeyChar == 8))
117      {
118          e.Handled = true; // bloquea todos los caratenres que no cumplen con la condicion
119      }
120  }
121
122  1 referencia
123  private void txtPasware_KeyPress(object sender, KeyPressEventArgs e)
124  {
125      if (!(e.KeyChar >= 97 && e.KeyChar <= 122
126          || e.KeyChar == 8 || (e.KeyChar >= 47 && e.KeyChar <= 57)))
127      {
128          e.Handled = true; // bloquea todos los caratenres que no cumplen con la condicion
129      }
130  }

```

Ilustración 29 Código Ventana Form1

The image shows a screenshot of a graphical user interface window titled "Form1". The window has a standard Windows-style title bar with minimize, maximize, and close buttons. Inside the window, there are two text input fields. The first field is labeled "Usuario" and the second is labeled "Pasware". Below these fields, there are three buttons: "Aceptar", "Guarda", and "Salir". The window is set against a light gray background.

Ilustración 30 Diseño Ventana Form1

```

12 namespace practica_1
13 {
14     6 referencias
15     public partial class crud : Form
16     {
17         // Variables para guardar la celda seleccionada / Variables to store selected cell
18         int _idCeldaSelec = -1, _idColumnaSelec = -1;
19
20         // Guardamos el usuario actual / Store current user
21         string _usuario = "";
22
23         2 referencias
24         public crud() // constructor de la clase
25         {
26             InitializeComponent();
27             precargaDeDatos();
28
29             // Cargamos los datos desde el archivo / Load data from file
30             2 referencias
31             private void precargaDeDatos()
32             {
33                 dataUsuario_crud.Rows.Clear();
34                 int contRegist = 0;
35
36                 // Abrimos el archivo csv / Open csv file
37                 StreamReader leerArchivo = File.OpenText("usuario.csv");
38                 string datoExtraido = "";
39
40                 do
41                 {
42                     datoExtraido = leerArchivo.ReadLine();
43
44                     if (datoExtraido != null)
45                     {
46                         // Separamos los datos / Split data
47                         string[] datos = datoExtraido.Split(" ", );
48                         contRegist++;
49
50                         // Agregamos datos a la tabla / Add data to table
51                         dataUsuario_crud.Rows.Add(contRegist.ToString(), datos[0], datos[1]);
52                     }
53                 } while (datoExtraido != null); // se repite mientras el renglon no este bacio
54
55                 // Cerramos el archivo / Close file
56                 leerArchivo.Close();
57             }
58
59             1 referencia
60             private void label1_Click(object sender, EventArgs e)
61             {
62             }
63
64             1 referencia
65             private void btnSalir_crud_Click(object sender, EventArgs e)
66             {
67                 Application.Exit(); // forzamos a cerrar todo el programa
68             }
69
70             1 referencia
71             private void dataUsuario_crud_MouseClick(object sender, MouseEventArgs e)
72             {
73                 //MessageBox.Show("Se ha actua por el click");
74             }
75
76             1 referencia
77             private void dataUsuario_crud_CellClick(object sender, DataGridViewCellEventArgs e)
78             {
79                 //MessageBox.Show("Se ha actua por el click en la celda");
80
81                 // Guardamos la fila y columna seleccionada / Save selected row and column
82                 _idCeldaSelec = e.RowIndex;
83                 _idColumnaSelec = e.ColumnIndex;
84
85                 if (_idCeldaSelec != -1)
86                 {
87                     // Extraemos datos de la tabla / Extract data from table
88                     string idT = dataUsuario_crud.Rows[_idCeldaSelec].Cells[0].Value.ToString(); // del valor de la tanbla seleeciona el
89                     string usuarioT = dataUsuario_crud.Rows[_idCeldaSelec].Cells[1].Value.ToString();
90                     string paswareT = dataUsuario_crud.Rows[_idCeldaSelec].Cells[2].Value.ToString();
91
92                     // Guardamos el usuario seleccionado / Store selected user
93                     _usuario = usuarioT;
94
95                     // Mostramos datos en textbox / Show data in textbox
96                     txtUsuario_crud.Text = usuarioT;
97                 }
98             }
99         }
100     }

```



```

95         txtPasware_crud.Text = paswareT;
96
97         // Activamos botones / Enable buttons
98         btnEliminar_crud.Enabled = true;
99         btnAcivar_crud.Enabled = true;
100     }
101
102
103     1 referencia
104     private void btnAcivar_crud_Click(object sender, EventArgs e)
105     {
106         // Mostramos botones de edición / Show edit buttons
107         btnModificar_crud.Visible = true;
108         btnEliminar_crud.Visible = false;
109         btnAcivar_crud.Visible = false;
110
111         // Activamos los textbox / Enable textboxes
112         txtUsuario_crud.Enabled = true;
113         txtPasware_crud.Enabled = true;
114     }
115
116     1 referencia
117     private void btnEliminar_crud_Click(object sender, EventArgs e)
118     {
119         // Validamos selección / Validate selection
120         if (_idCeldaSelec == -1)
121         {
122             return;
123         }
124
125         // Eliminamos fila seleccionada / Remove selected row
126         dataUsuario_crud.Rows.RemoveAt(_idCeldaSelec); // eliminamos un elemento de la tabla
127
128         // Desactivamos botones / Disable buttons
129         btnEliminar_crud.Enabled = false;
130         btnAcivar_crud.Enabled = false;
131
132         // Reiniciamos variables / Reset variables
133         _idCeldaSelec = -1; // recetamos la variable
134
135         // Eliminamos del archivo / Delete from file
136         EliminarDelArchivo();
137
138         // Limpiamos cajas de texto / Clear textboxes
139         txtPasware_crud.Clear();
140         txtUsuario_crud.Clear();
141
142         // Eliminamos usuario del archivo / Delete user from file
143         1 referencia
144         private void EliminarDelArchivo()
145         {
146             string usuarioName = txtUsuario_crud.Text;
147             string datoExtraido = "";
148
149             // Abrimos archivos / Open files
150             StreamReader lectorArchivo = File.OpenText("usuario.csv");
151             StreamWriter auxiliar = File.AppendText("temp.csv");
152
153             do
154             {
155                 datoExtraido = lectorArchivo.ReadLine();
156
157                 if (datoExtraido != null)
158                 {
159                     // Dividimos datos / Split data
160                     string[] datos = datoExtraido.Split(" ", StringSplitOptions.RemoveEmptyEntries);
161
162                     // Verificamos usuario / Verify user
163                     if (usuarioName.Equals(datos[0]))
164                     {
165                         // Guardamos datos temporales / Save temp data
166                         auxiliar.WriteLine(datoExtraido);
167                     }
168                 }
169             } while (datoExtraido != null); // se repite mientras el renglon no este bacio
170
171             // Cerramos archivos / Close files
172             auxiliar.Close();
173             lectorArchivo.Close();
174
175             // Reemplazamos archivo original / Replace original file
176             File.Delete("usuario.csv");
177             File.Move("temp.csv", "usuario.csv"); // cambiamos el nombre al archivo auxiliar
178         }
179     }

```

```

182         MessageBox.Show("El usuario ha sido eliminado ☹️");
183     }
184
185     // Buscamos coincidencias / Search matches
186     1 referencia
187     public bool Coincidencias(string datosBusqueda, string nombreArchivo = "usuario.csv")
188     {
189         try
190         {
191             // Abrimos archivo / Open file
192             StreamReader lectorArchivo = File.OpenText(nombreArchivo);
193             string datosTem = "";
194
195             //bool estado = false;
196             do
197             {
198                 datosTem = lectorArchivo.ReadLine();
199
200                 if (datosTem != null)
201                 {
202                     // Dividimos datos / Split data
203                     string[] datoB = datosTem.Split(" ", "");
204
205                     // Validamos coincidencia / Validate match
206                     if (datoB[0].Equals(datosBusqueda))
207                     {
208                         //estado = true;
209
210                         // Cerramos archivo / Close file
211                         lectorArchivo.Close();
212                         return true;
213                     }
214                 }
215             } while (datosTem != null);
216
217             // Cerramos archivo / Close file
218             lectorArchivo.Close();
219             return false;
220         } catch (Exception ex)
221         {
222             // Mostramos error en consola / Show error in console
223             Debug.WriteLine("Error: " + ex.Message);
224             return false;
225         }
226     }
227
228     1 referencia
229     private void btnModificar_crud_Click(object sender, EventArgs e)
230     {
231         // Validamos selección / Validate selection
232         if (_idCeldaSelec == -1)
233         {
234             return;
235         }
236
237         // Modificamos usuario / Modify user
238         modificarUsuario();
239
240         // Recargamos datos / Reload data
241         precargaDeDatos();
242
243         // Limpiamos textbox / Clear textboxes
244         txtPasware_crud.Clear();
245         txtUsuario_crud.Clear();
246
247         // Desactivamos edición / Disable editing
248         txtPasware_crud.Enabled = false;
249         txtUsuario_crud.Enabled = false;
250
251         //btnModificar_crud.Enabled = false;
252
253         // Restauramos botones / Restore buttons
254         btnEliminar_crud.Enabled = false;
255         btnEliminar_crud.Visible = true;
256         btnModificar_crud.Visible = false;
257         btnAcivar_crud.Visible = true;
258         btnAcivar_crud.Enabled = false;
259     }
260
261     // Modificamos usuario en archivo / Modify user in file
262     1 referencia
263     private void modificarUsuario()
264     {
265     }
266 
```

```

267 string usuarioName = txtUsuario_crud.Text;
268 string pasware = txtPasware_crud.Text;
269
270 //if (!Coincidencias(usuarioName))
271 //{
272 //    MessageBox.Show("El usuario ya esta registrado");
273 //    return;
274 //}
275
276 string datoExtraido = "";
277
278 // Abrimos archivos / Open files
279 StreamReader lectorArchivo = File.OpenText("usuario.csv");
280 StreamWriter auxiliar = File.AppendText("temp.csv");
281
282 do
283 {
284     datoExtraido = lectorArchivo.ReadLine();
285
286     if (datoExtraido != null)
287     {
288         // Separamos datos / Split data
289         string[] datos = datoExtraido.Split(" ", " ");
290
291         // Validamos usuario / Validate user
292         if (_usuario.Equals(datos[0])) // si el usuario es igual al registrado
293         {
294             // Guardamos datos modificados / Save modified data
295             auxiliar.WriteLine(usuarioName + " " + pasware); // guardamos los nuevos datos en el temporal
296         }
297         else
298         {
299             // Copiamos datos sin cambios / Copy unchanged data
300             auxiliar.WriteLine(datoExtraido); // el resto lo copiamos igual en el temporal
301         }
302     }
303 }
304 while (datoExtraido != null); // se repite mientras el renglon no este vacio
305
306 // cerramos los archivos
307 // Close files
308 auxiliar.Close();
309 lectorArchivo.Close();
310
311 // Reemplazamos archivo original / Replace original file
312 File.Delete("usuario.csv");
313 File.Move("temp.csv", "usuario.csv"); // cambiamos el nombre al archivo auxiliar
314
315 MessageBox.Show("El usuario fue Modificado 😊", "Modificar Usuario");
316
317
318
319

```

Ilustración 31 Código de la ventana CRUD

N°	Usuario	Contraseña
*		

Ilustración 32 Diseño de Ventana Crud

Form1

Usuario

Pasware

Aceptar Guarda Salir

crud

Usuario

Pasware

Eliminar Activar Salir

	N°	Usuario	Contraseña
▶	1	invitado	1235
	2	usuario	123456
	3	admin	12345
*			

Ilustración 33 Ejecución del Programa

Unidad 5

Practica 1 (Videojuego de Naves Espaciales)

Descripción: Con la intención de aprender como se controlan los archivos en los videojuegos, desarrollamos un videojuego que guarda en un archivo el puntaje del juegos a lo largo de su partida, en su partida debe pasar por varios niveles y derrotar un numero determinado de enemigos, cuando pierde puede guardar su progreso y este registro será almacenado en un archivo de puntajes generales, esto permite que múltiples jugadores puedan guardar y retomar sus puntajes.

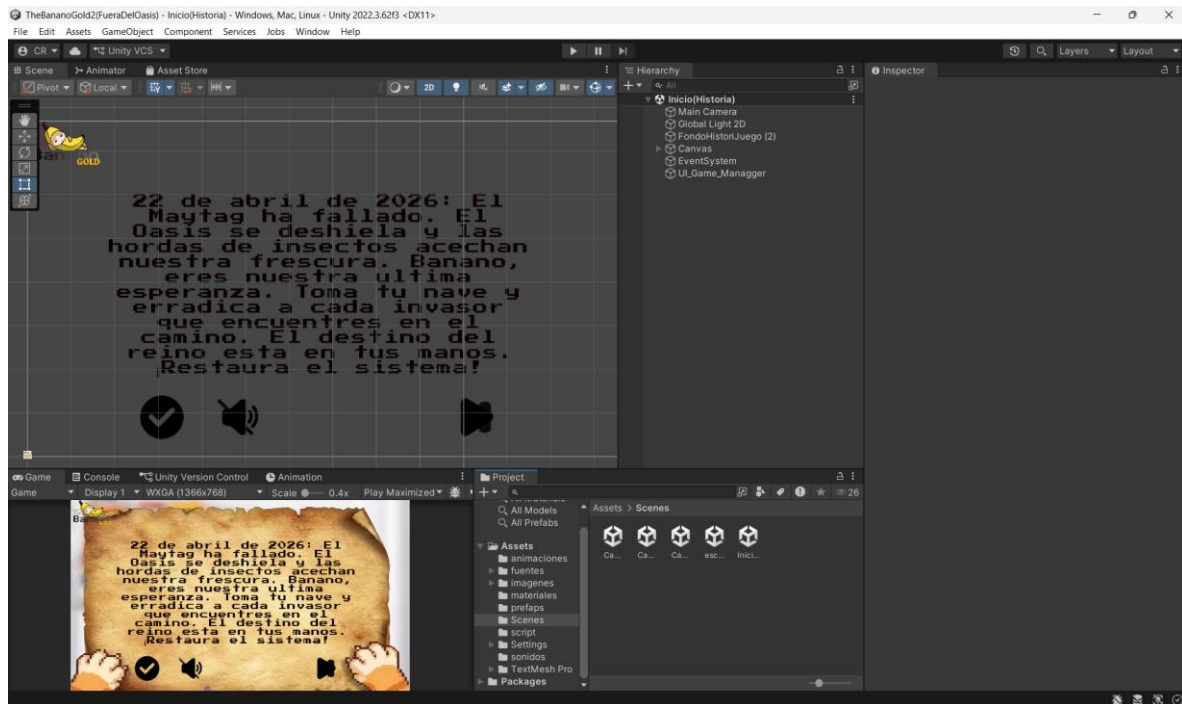


Ilustración 34 Diseño de la escena que contienen la historia

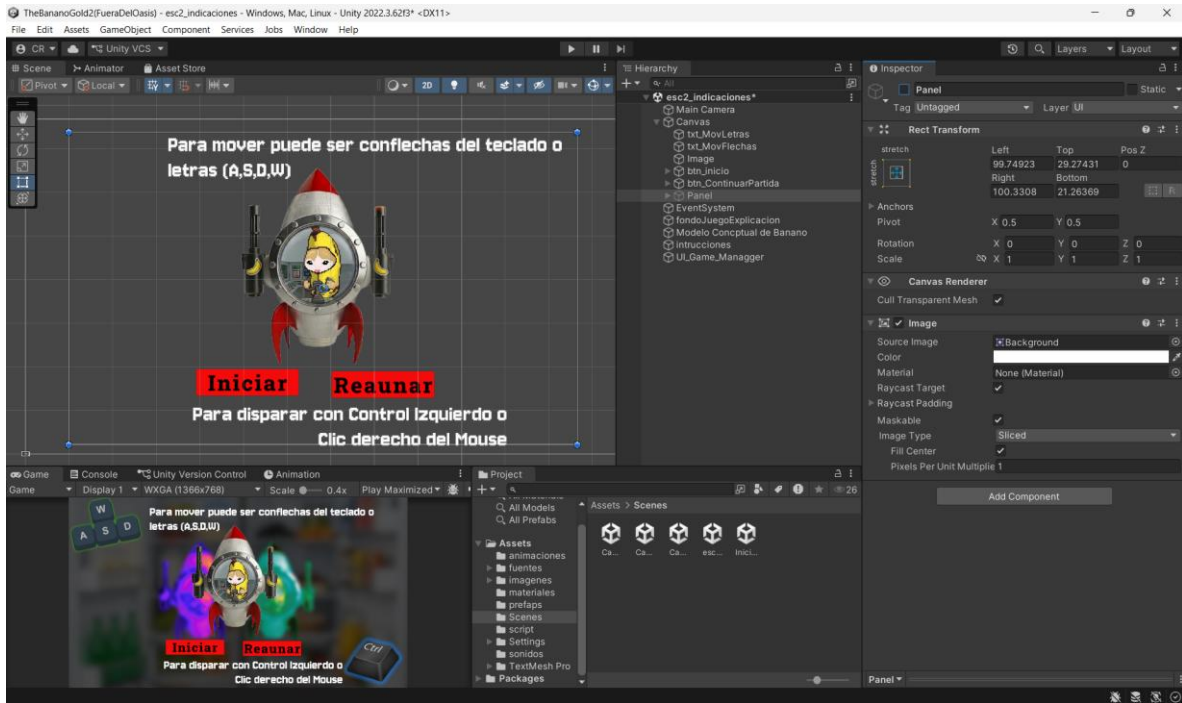


Ilustración 35 Diseño de la Escena que contiene las Indicaciones de jugabilidad

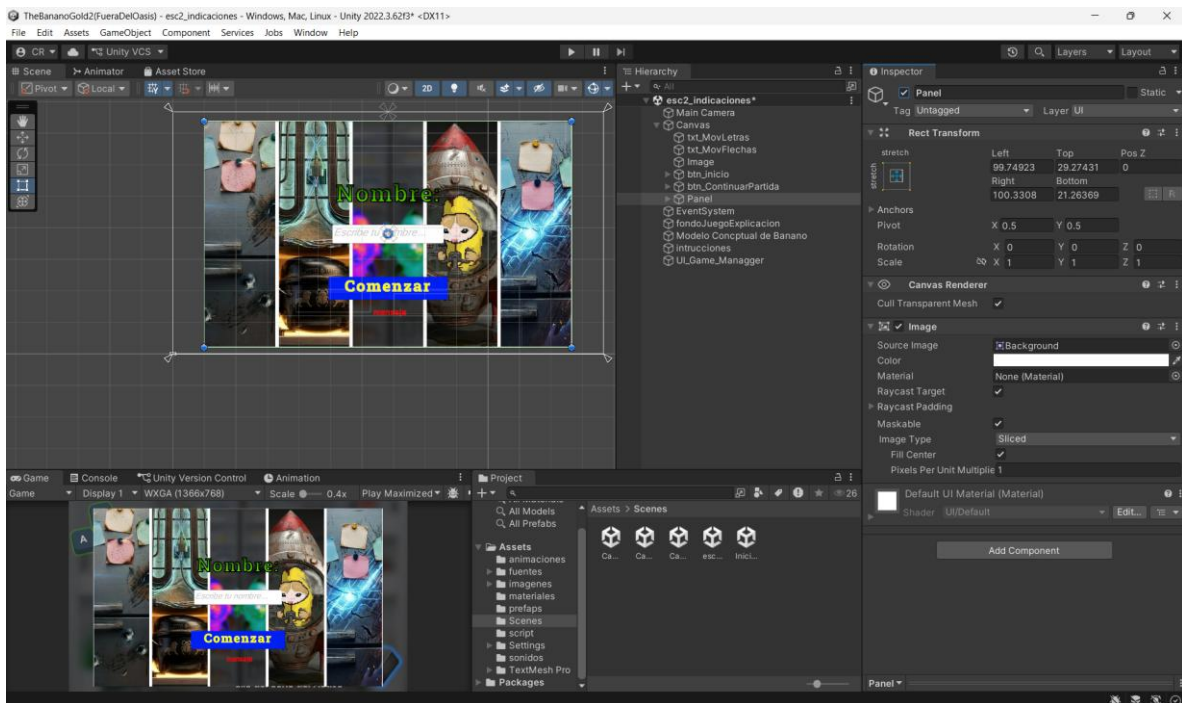


Ilustración 36 Diseño del Panel Para Retomar Partida

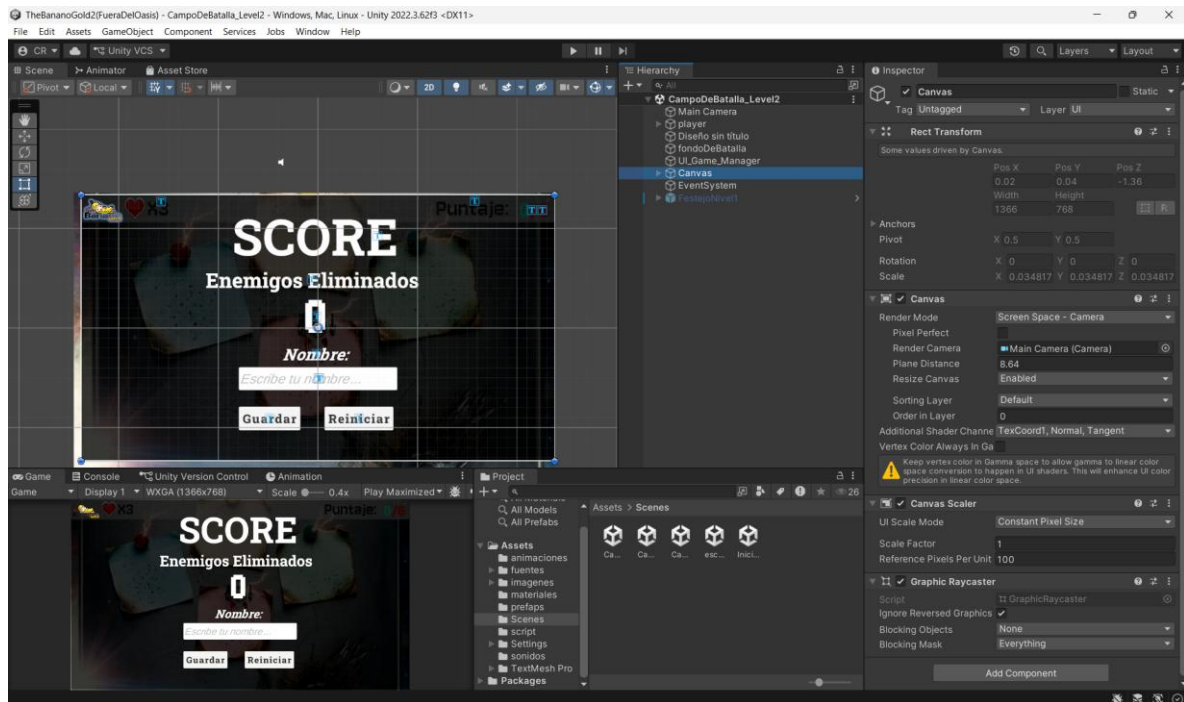


Ilustración 37 Diseño del Panel Para Guardar Partida

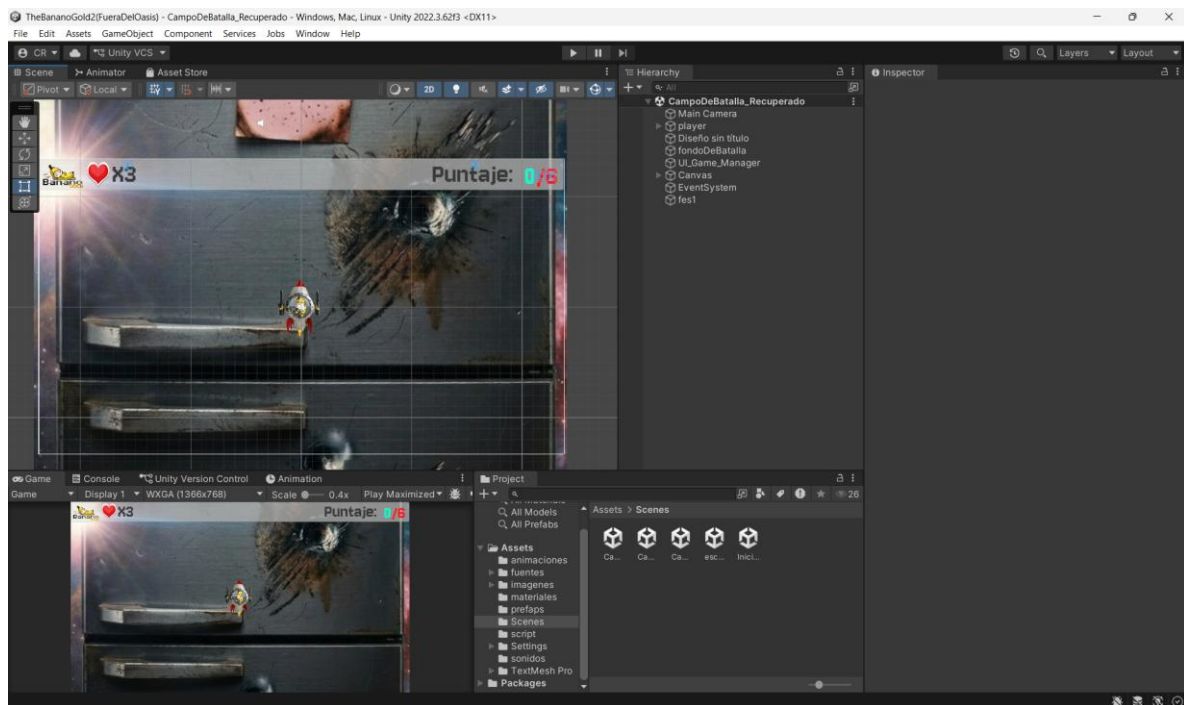


Ilustración 38 Diseño de escena de combate Nivel 1

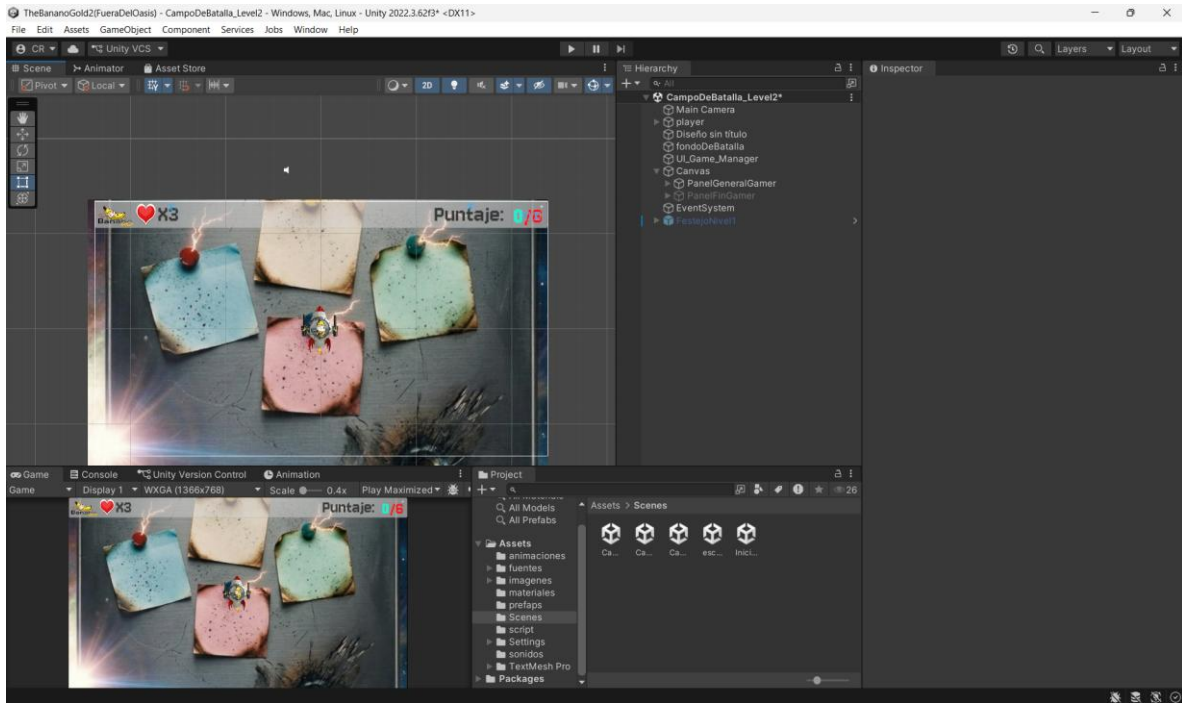


Ilustración 39 Diseño de escena de combate Nivel 2

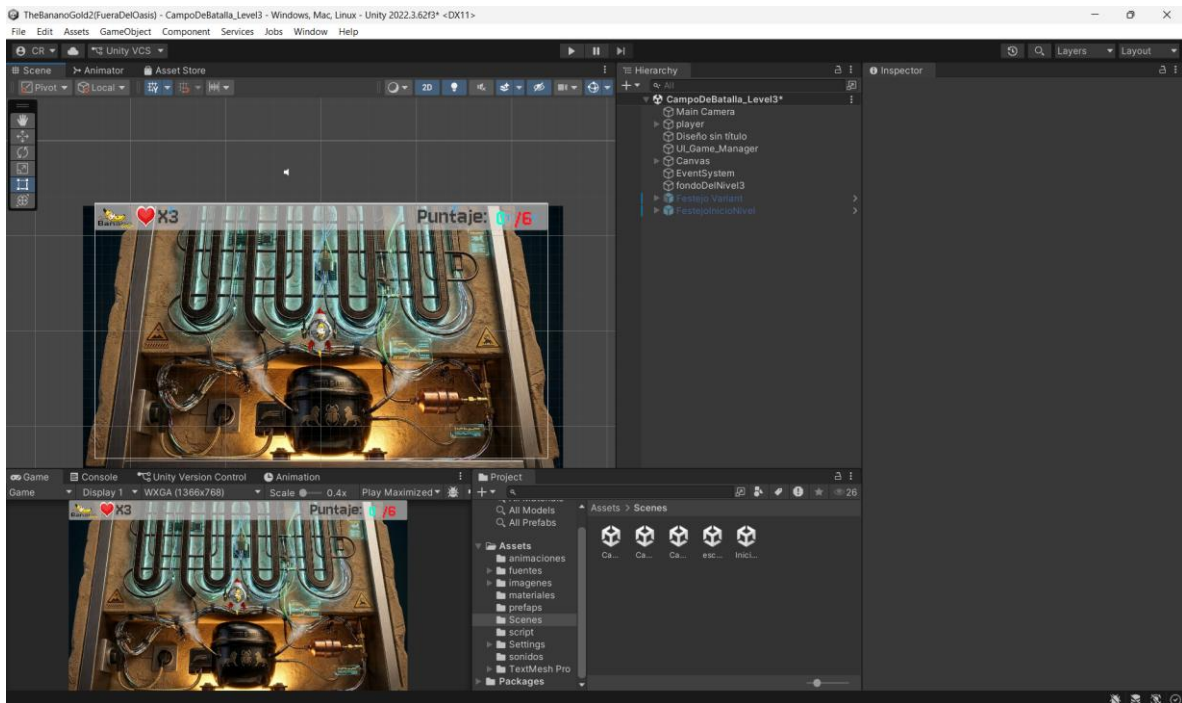


Ilustración 40 Diseño de escena de combate Nivel 3


```

7      {
8      public AudioSource voz_historia;
9      public GameObject btn_play;
10     public GameObject btn_pausar;
11     public GameObject btn_sonido;
12     public GameObject btn_sinSonido;
13     //private bool estVol = false;
14     // Start is called before the first frame update
15     @ Mensaje de Unity | 0 referencias
16     void Start()
17     {
18         voz_historia.Play();
19         voz_historia.Pause();
20
21         btn_pausar.SetActive(false);
22     }
23
24     // Update is called once per frame
25     @ Mensaje de Unity | 0 referencias
26     void Update()
27     {
28     }
29     0 referencias
30     public void Reproducir_voz()
31     {
32         voz_historia.UnPause();
33         btn_pausar.SetActive(true);
34         btn_play.SetActive(false);
35     }
36     0 referencias
37     public void Detener_voz()
38     {
39         voz_historia.Pause();
40         btn_pausar.SetActive(false);
41         btn_play.SetActive(true);
42     }
43     0 referencias
44     public void Silenciar()
45     {
46         voz_historia.mute = true;
47         btn_sonido.SetActive(false);
48         btn_sinSonido.SetActive(true);
49     }
50     0 referencias
51     public void activarVolumen()
52     {
53         voz_historia.mute = false;
54         btn_sonido.SetActive(true);
55         btn_sinSonido.SetActive(false);
56     }
57     }

```

Ilustración 41 Código del archivo UI

```
9 public class ControladorDeArchivos : MonoBehaviour
10 {
11     private const string nombreArchivo = "PuntajeGlobal.csv";
12     //METODO QUEGUARDA EL PUNTAJE DE JUGADOR EN UNA LISTA DE JUGADORES
13     1 referencia
14     public static void GuardarArchivoMultiple(string archivoN = nombreArchivo, string nombre = "invitado", int puntos = 0)
15     {
16         if (!(File.Exists(archivoN)) || !BuscarParametro(parametro: nombre))
17         {
18             //Console.WriteLine("El archivo no existe");
19             GuardarGen(puntos, nombre);
20             return;
21         }
22         StreamReader leerArchivo = null;
23         leerArchivo = File.OpenText(archivoN);
24         string datos;
25         do
26         {
27             datos = leerArchivo.ReadLine();
28             if (datos != null)
29             {
30                 string[] d = datos.Split(",");
31                 if (nombre.Equals(d[0]))
32                 {
33                     datos = d[0] + "," + puntos.ToString();
34                 }
35             }
36             Console.WriteLine("Copiando datos");
37             StreamWriter archivo = File.AppendText("datosAuxiliar.csv");
38             archivo.WriteLine(datos);
39             archivo.Close();
40         }
41         while (datos != null); // se repite hasta que el archivo regrese un valor null
42         leerArchivo.Close();
43         File.Delete(archivoN);
44         File.Copy("datosAuxiliar.csv", archivoN);
45         if (File.Exists("datosAuxiliar.csv"))
46         {
47             File.Delete("datosAuxiliar.csv");
48         }
49     }
50     //METODO PARA BUSCAR UN NOMBRE EN LA LISTA DE JUGADORES
51     2 referencias
52     public static bool BuscarParametro(string archivo = nombreArchivo, string parametro = "")
53     {
54     }
55 }
56
57
```

```
58     {
59         try
60         {
61             if (parametro == "") { return false; }
62             StreamReader archivoLect = File.OpenText(archivo);
63             string contArchivo = archivoLect.ReadToEnd();
64             archivoLect.Close();
65             if (Regex.IsMatch(contArchivo, $"\\b{Regex.Escape(parametro)}\\b"))
66             {
67                 return true;
68             }
69             return false;
70         }
71         catch (Exception e)
72         {
73             Debug.LogError("Error en Búsqueda: " + e);
74             return false;
75         }
76     }
77     1 referencia
78     public static void GuardarGen(int puntos = 0, string nom = "Invidatos", string archivo = nombreArchivo)
79     {
80         StreamWriter escribir;
81         escribir = File.AppendText(archivo);
82
83         escribir.WriteLine(nom + ", " + puntos.ToString());
84
85         escribir.Close();
86     }
87
88     //METODO PARA RETRONAR EL PUNTAJE DEL JUGADOR BUECADO
89     1 referencia
90     public static int EncontrarDato(string nom = "Invidatos", string archivoN = nombreArchivo)
91     {
92         try
93         {
94             StreamReader leerArchivo = null;
95             leerArchivo = File.OpenText(archivoN);
96             string datos;
97             do
98             {
99                 datos = leerArchivo.ReadLine();
100                 if (datos != null)
101                 {
102                     string[] d = datos.Split(",");
103                     if (nom.Equals(d[0]))
104                     {
105                         //Console.WriteLine("Si lo encontro");
106                         return int.Parse(d[1]);
107                     }
108                 }
109             } while (datos != null);
110         }
111         catch (Exception e)
112         {
113             Debug.LogError("Error en Búsqueda: " + e);
114             return -1;
115         }
116     }
```

```

106     }
107
108     }
109
110     }
111     while (datos != null); // se repite hasta que el archivo regrese un valor null
112     leerArchivo.Close();
113     return -1;
114 }
115 catch (Exception e)
116 {
117     Debug.LogError("Error al consultar dato: " + e);
118     return -1;
119 }
120
121 1 referencia
122 public static void EliminarDeSeccionMultiple(string archivoN = nombreArchivo, string nombre = "")
123 {
124     try
125     {
126         if (!File.Exists(archivoN))
127         {
128             //Console.WriteLine("El archivo no existe");
129             return;
130         }
131         StreamReader leerArchivo = null;
132         leerArchivo = File.OpenText(archivoN);
133         string datos;
134         do
135         {
136             datos = leerArchivo.ReadLine();
137             if (datos != null)
138             {
139                 string[] d = datos.Split(",");
140                 if (nombre.Equals(d[0]))
141                 {
142                     Console.WriteLine("Copiando datos");
143                     StreamWriter archivo = File.AppendText("datosAuxiliar.csv");
144                     archivo.WriteLine(datos);
145                     archivo.Close();
146                 }
147             }
148         } while (datos != null); // se repite hasta que el archivo regrese un valor null
149         leerArchivo.Close();
150         File.Delete(archivoN);
151
152         File.Copy("datosAuxiliar.csv", archivoN);
153         if (File.Exists("datosAuxiliar.csv"))
154         {
155             File.Delete("datosAuxiliar.csv");
156         }
157     } catch (Exception e)
158     {
159         Debug.LogError(e);
160     }
161 }
162
163
164
165
166
167
168

```

Ilustración 42 Códigos del archivo Controlador de archivos

```

10 public class contadorEnemigos : MonoBehaviour
11 {
12     public static int contadorEnEliminados = 0;
13     public TextMeshProGUI puntuacion;
14     public TextMeshProGUI vidas;
15     public static int contVidas = 3;
16     public GameObject panel;
17     public TextMeshProGUI txt_enemigosEliminaso;
18     public TextMeshProGUI txt_labelPuntosMeta;
19     public int PuntosDeMeta = 6;
20     public GameObject celebracionNivel;
21     public GameObject personaje;
22     public AudioSource musicaDerrota;
23     public static string nom = "invitado";
24
25
26     //public InputField input_nombre;
27     public TMP_InputField input_nombre;
28     // Start is called before the first frame update
29     // Mensaje de Unity | 0 referencias
30     private void Awake()
31     {
32         StreamReader leer;
33         if (File.Exists("Puntaje.csv"))
34         {
35             leer = File.OpenText("Puntaje.csv");
36             string Datos = leer.ReadLine();
37             string[] p = Datos.Split(",");
38             contadorEnEliminados = int.Parse(p[1]);
39             leer.Close();
40         }
41     }
42
43
44
45     0 referencias
46     public void Guardar()
47     {
48         //GuardarGen();
49
50         //GuardarGen();
51         string nomT = input_nombre.text;
52         if (nomT.Length == 0) { nomT = "invitado"; }
53         ControladorDeArchivos.GuardarArchivoMultiple(nombre: nomT , puntos: contadorEnEliminados);
54         File.Delete("Puntaje.csv");
55         panel.gameObject.SetActive(false);
56         Application.Quit();
57
58     }
59     2 referencias
60     public void GuardarGen()
61     {
62         File.Delete("Puntaje.csv");
63         StreamWriter escribir;
64         escribir = File.AppendText("Puntaje.csv");
65         nom = input_nombre.text;
66         if (nom == null)
67         {
68             nom = "invitado";
69         }
70         escribir.WriteLine(nom + "," + contadorEnEliminados.ToString());
71         //escribir.WriteLine(nom + " , " + contadorEnEliminados.ToString());
72         escribir.Close();
73     }
74     // Mensaje de Unity | 0 referencias
75     void Start()
76     {
77         panel.gameObject.SetActive(false);
78         celebracionNivel.SetActive(false);
79         txt_labelPuntosMeta.text = "/" + PuntosDeMeta.ToString();
80
81         //if (SceneManager.GetActiveScene().buildIndex == 3)
82         //{
83             StartCoroutine(celebrarInicioNivel(2.0f));
84         //}
85     }

```

```

95 void Update()
96 {
97     vidas.text = "X" + contadorEnemigos.contVidas.ToString();
98     puntuacion.text = contadorEnemigos.contadorEnEliminados.ToString();
99     if (contVidas <= 0)
100     {
101         //print("no tienes vidas");
102         musicaDerrota.Play();
103         Time.timeScale = 0;
104         panel.SetActive(true);
105         print("se activo el panel");
106         txt_enemigosEliminados.text = contadorEnemigos.contadorEnEliminados.ToString();
107         //Guardar(true);
108         return;
109     }
110     else if (contadorEnEliminados >= PuntosDeMeta)
111     {
112         // hacer animacion de la nave creciendo y volando
113
114         //celebracion sigEsc = new celebracion();
115         //sigEsc.InciarInterrupcion();
116         int proximaEscena = SceneManager.GetActiveScene().buildIndex + 1;
117         if (proximaEscena < SceneManager.sceneCountInBuildSettings)
118         {
119             GuardarGen();
120             SceneManager.LoadScene(proximaEscena);
121         }
122         else
123         {
124             File.Delete("Puntaje.csv");
125             ControladorDeArchivos.EliminarDeSeccionMultiple(nombre: nom);
126             celebracionNivel.SetActive(true);
127             personaje.SetActive(false);
128         }
129     }
130     print(contadorEnEliminados);
131 }
132
133 2 referencias
134 public static void IncrementarEnemigo()
135 {
136     contadorEnEliminados ++;
137 }
138
139 2 referencias
140 public static void DescontarVidas()
141 {
142     contVidas--;
143 }
144
145 0 referencias
146 public void reiniciar()
147 {
148     GuardarGen();
149     contVidas = 3;
150     Time.timeScale = 1;
151     SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
152     panel.SetActive(false);
153 }

```

Ilustración 43 Código del Archivo Contador de Enemigos

```

7 public class ReanudarJuego : MonoBehaviour
8 {
9     public TMP_InputField input_nombre;
10    public GameObject panelReanudar;
11    public TextMeshProUGUI txt_mensaje;
12    // Start is called before the first frame update
13    @ Mensaje de Unity | 0 referencias
14    void Start()
15    {
16        panelReanudar.SetActive(false);
17        txt_mensaje.text = "";
18    }
19    // Update is called once per frame
20    @ Mensaje de Unity | 0 referencias
21    void Update()
22    {
23    }
24    0 referencias
25    public void ReanudarPartida()
26    {
27        string nom = input_nombre.text;
28        if (!ControladorDeArchivos.BuscarParametro(parametro:nom))
29        {
30            nom = "invitado";
31            txt_mensaje.text = "Accedio \n como \n invitado";
32        }
33        else { txt_mensaje.text = "CARGANDO ..."; }
34        contadorEnemigos.contadorEnEliminados = ControladorDeArchivos.EncontrarDato(nom);
35        //panelReanudar.SetActive(false);
36        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex + 1);
37    }
38    0 referencias
39    public void ActivarPanel()
40    {
41        panelReanudar.SetActive(true);
42    }
43    }
44    }
45    }
46

```

Ilustración 44 Código del Archivo Reanudar Juego

```

5 @ Script de Unity (1 referencia de recurso) | 0 referencias
6 public class EliminarAdistancia : MonoBehaviour
7 {
8     // Start is called before the first frame update
9     @ Mensaje de Unity | 0 referencias
10    void Start()
11    {
12    }
13    // Update is called once per frame
14    @ Mensaje de Unity | 0 referencias
15    void Update()
16    {
17    }
18    @ Mensaje de Unity | 0 referencias
19    private void OnTriggerEnter2D(Collider2D collision)
20    {
21        if (collision.CompareTag("bala"))
22        {
23            contadorEnemigos.IncrementarEnemigo();
24            Destroy(collision.gameObject);
25        }
26        else if (collision.CompareTag("Player"))
27        {
28            contadorEnemigos.DescontarVidas();
29        }
30        else
31        {
32            return;
33        }
34        Destroy(gameObject);
35    }
36    }
37

```

Ilustración 45 Código del Archivo Eliminar Enemigo

```

5 public class disparar : MonoBehaviour
6 {
7     public float velocidad_Bala = 50f;
8     public GameObject bala_ob;
9     public Transform armaP;
10    bool est = true;
11    public float frecuenciaDeDisparo = 5.0f;
12    // Start is called before the first frame update
13    // Mensaje de Unity | 0 referencias
14    void Start()
15    {
16    }
17
18    // Update is called once per frame
19    // Mensaje de Unity | 0 referencias
20    void Update()
21    {
22        if (est)
23        {
24            StartCoroutine(temporizador(frecuenciaDeDisparo));
25        }
26    }
27
28    // 1 referencia
29    private void Disparar(Transform arma)
30    {
31        GameObject clonBala = Instantiate(bala_ob, arma.position, arma.rotation);
32        Rigidbody2D cuerpoBala = clonBala.GetComponent<Rigidbody2D>();
33        cuerpoBala.velocity = arma.up * velocidad_Bala;
34        Destroy(clonBala, 5f);
35    }
36    // 1 referencia
37    IEnumerator temporizador(float tiempo)
38    {
39        Disparar(armaP);
40        est = false;
41        yield return new WaitForSeconds(tiempo);
42        est = true;
43    }

```

Ilustración 46 Código del Archivo Disparar

```

6 public class celebracionInicio : MonoBehaviour
7 {
8     public GameObject celebracionNivel;
9     public GameObject personaje;
10    // Start is called before the first frame update
11    // Mensaje de Unity | 0 referencias
12    void Start()
13    {
14        if (SceneManager.GetActiveScene().buildIndex <= 4 && SceneManager.GetActiveScene().buildIndex >= 2)
15        {
16            StartCoroutine(celebrarInicioNivel(2.0f));
17        }
18    }
19    // 1 referencia
20    IEnumerator celebrarInicioNivel(float tiempo)
21    {
22        celebracionNivel.SetActive(true);
23        personaje.SetActive(false);
24        yield return new WaitForSeconds(tiempo);
25        celebracionNivel.SetActive(false);
26        personaje.SetActive(true);
27    }
28
29    // Update is called once per frame
30    // Mensaje de Unity | 0 referencias
31    void Update()
32    {
33    }

```

Ilustración 47 Código del Archivo Celebración De Inicio


```

5 public class enemigo : MonoBehaviour
6 {
7     public float velocidad = 10f;
8     public float velocidadPersecucion = 8f;
9     Vector2 posicion;
10    public float distancia = 200f;
11    public bool seguir_Personaje = false;
12    public Transform objetivo;
13    float timer;
14
15    // Start is called before the first frame update
16    // Mensaje de Unity | 0 referencias
17    void Start()
18    {
19        posicion = transform.position;
20        objetivo = GameObject.FindGameObjectWithTag("Player").transform;
21    }
22
23    // Update is called once per frame
24    // Mensaje de Unity | 0 referencias
25    void Update()
26    {
27        if (seguir_Personaje)
28        {
29            seguirPersonaje();
30            return;
31        }
32        transform.Translate(Vector3.down * velocidad * Time.deltaTime);
33        float inicio = Vector2.Distance(posicion, transform.position);
34        inicio = Mathf.Abs(inicio);
35        if (inicio > distancia)
36        {
37            Destroy(gameObject);
38        }
39    }
40
41    1 referencia
42    public void seguirPersonaje()
43    {
44        if (objetivo == null) return;
45
46        Vector3 direccion = (objetivo.position - transform.position).normalized;
47        transform.position += direccion * velocidadPersecucion * Time.deltaTime;
48        transform.up = - (direccion);
49        timer += Time.deltaTime;
50    }

```

```

50 0 referencias
51 public void destruirPorDistancia()
52 {
53 }
54
55 // Mensaje de Unity | 0 referencias
56 private void OnTriggerEnter2D(Collider2D collision)
57 {
58     if (collision.CompareTag("bala"))
59     {
60         contadorEnemigos.IncrementarEnemigo();
61         Destroy(collision.gameObject);
62     }
63     else if (collision.CompareTag("Player"))
64     {
65         contadorEnemigos.DescontarVidas();
66     }
67     else
68     {
69         return;
70     }
71     Destroy(gameObject);
72 }
73
74

```

Ilustración 48 Código Del Archivo Enemigos

```

5 public class Bala : MonoBehaviour
6 {
7     public float velocidad_Bala = 50f;
8     public GameObject bala_ob;
9     //public Transform armaIzq;
10    //public Transform armaDer;
11    public Transform[] naveNivel0;
12    public Transform[] naveNivel1;
13    public Transform[] naveNivelDanada;
14
15    public AudioSource audioBala;
16
17    public Sprite spriteNivel0;
18    public Sprite spriteNivel1;
19    public Sprite spriteNivelDanada;
20
21    public SpriteRenderer sr;
22
23    //private bool estModoNave = false;
24    private int modoNave = 2;
25
26    private float tiempoPoder = 5.0f;
27    // Start is called before the first frame update
28    // Mensaje de Unity | 0 referencias
29    void Start()
30    {
31        sr = GetComponent<SpriteRenderer>();
32        ActivarArmaN0();
33    }
34
35    // Update is called once per frame
36    // Mensaje de Unity | 0 referencias
37    void Update()
38    {
39        if (Input.GetButtonDown("Fire1"))
40        {
41            shoot();
42            audioBala.Play();
43        }
44    }
45    // 1 referencia
46    private void shoot()
47    {
48        //Transform[] actuales = !estModoNave ? naveNivel0 : naveNivel1;
49        Transform[] actuales;
50        if (modoNave == 1)
51        {
52            actuales = naveNivelDanada;
53        }
54    }

```

```

51     }
52     else if (modoNave == 3)
53     {
54         actuales = naveNivel1;
55     }
56     else
57     {
58         actuales = naveNivel0;
59     }
60     foreach (Transform t in actuales)
61     {
62         Disparar(t);
63     }
64 }
65 1 referencia
66 private void Disparar(Transform arma)
67 {
68     GameObject clonBala = Instantiate(bala_ob, arma.position, arma.rotation);
69     Rigidbody2D cuerpoBala = clonBala.GetComponent<Rigidbody2D>();
70     cuerpoBala.velocity = arma.up * velocidad_Bala;
71     Destroy(clonBala, 5f);
72 }
73 1 referencia
74 private void ActivarArmaN0()
75 {
76     //estModoNave = false;
77     modoNave = 2;
78     sr.sprite = spriteNivel0;
79 }
80 1 referencia
81 public void ActivarNaveDanada()
82 {
83     //estModoNave = false;
84     modoNave = 1;
85     sr.sprite = spriteNivelDanada;
86 }
87 1 referencia
88 private void OnTriggerEnter2D(Collider2D collision)
89 {
90     if (collision.CompareTag("poder"))
91     {
92         ActivarPoder(tiempoPoder);
93         Destroy(collision.gameObject);
94     }
95     else if (collision.CompareTag("enemigo") && modoNave != 3)
96     {
97         ActivarNaveDanada();
98     }
99 }
100 1 referencia
101 private void ActivarPoder(float duracion)
102 {
103     //estModoNave = true;
104     //sr.sprite = spriteNivel1;
105     StopAllCoroutines();
106     StartCoroutine(poderPersonaje(duracion));
107 }
108 1 referencia
109 IEnumerator poderPersonaje(float duracion)
110 {
111     //estModoNave = true;
112     modoNave = 3;
113     sr.sprite = spriteNivel1;
114     yield return new WaitForSeconds(duracion);
115     //estModoNave = false;
116     modoNave = 2;
117     sr.sprite = spriteNivel0;
118 }

```

Ilustración 49 Código Del Archivo Bala

```

5  public class audio : MonoBehaviour
6  {
7      public AudioSource musica;
8
9      // Start is called before the first frame update
10     // Mensaje de Unity | 0 referencias
11     void Start()
12     {
13         musica.Play();
14     }
15
16     // Update is called once per frame
17     // Mensaje de Unity | 0 referencias
18     void Update()
19     {
20     }
21 }

```

Ilustración 50 Código del Archivo Audio

```

5  public class creadorEnemigos : MonoBehaviour
6  {
7      public GameObject enemigos;
8      public float tiempo_aparicion = 1.0f;
9
10     // Start is called before the first frame update
11     // Mensaje de Unity | 0 referencias
12     void Start()
13     {
14         StartCoroutine(GenerarPoder());
15     }
16
17     // Update is called once per frame
18     // Mensaje de Unity | 0 referencias
19     void Update()
20     {
21     }
22
23     1 referencia
24     IEnumerator GenerarPoder()
25     {
26         while (true)
27         {
28             crear();
29             yield return new WaitForSeconds(tiempo_aparicion);
30         }
31     }
32
33     1 referencia
34     private void crear()
35     {
36         if (enemigos == null)
37         {
38             return;
39         }
40         Camera cam = Camera.main;
41         float alto = cam.orthographicSize;
42         float ancho = alto * cam.aspect;
43         float coordX = Random.Range(-ancho, ancho);
44         float coordY = (Random.value > 0.5f) ? alto + 1 : -alto - 1;
45         Vector3 posicion = new Vector3(coordX, 9.2f, -1);
46         GameObject clon = Instantiate(enemigos, posicion, Quaternion.identity);
47         Destroy(clon, 10.0f);
48     }
49 }

```

Ilustración 51 Código del Archivo Creador Enemigos

```

5 public class CrearPoderes : MonoBehaviour
6 {
7     public GameObject poder;
8     public float tiempo_aparicion = 1.0f;
9     public float distancia = 200f;
10
11     // Start is called before the first frame update
12     // Mensaje de Unity | 0 referencias
13     void Start()
14     {
15         StartCoroutine(GenerarPoder());
16     }
17
18     // Update is called once per frame
19     // Mensaje de Unity | 0 referencias
20     void Update()
21     {
22     }
23
24     1 referencia
25     IEnumerator GenerarPoder()
26     {
27         while(true)
28         {
29             crear();
30             yield return new WaitForSeconds(tiempo_aparicion);
31         }
32     }
33
34     1 referencia
35     private void crear()
36     {
37         if (poder == null)
38         {
39             return;
40         }
41
42         Camera cam = Camera.main;
43         float alto = cam.orthographicSize;
44         float ancho = alto * cam.aspect;
45         float coordX = Random.Range(-ancho, ancho);
46         float coordY = (Random.value > 0.5f) ? alto + 1 : -alto - 1;
47         Vector3 posicion = new Vector3(coordX, 9.2f, -1);
48         GameObject clon = Instantiate(poder, posicion, Quaternion.identity);
49         Destroy(clon, 5.0f);
50     }
51 }

```

Ilustración 52 Código del Archivo Creador de Poder

```

5 public class ElimiarAdistancia : MonoBehaviour
6 {
7     // Start is called before the first frame update
8     // Mensaje de Unity | 0 referencias
9     void Start()
10     {
11     }
12
13     // Update is called once per frame
14     // Mensaje de Unity | 0 referencias
15     void Update()
16     {
17     }
18
19     // Mensaje de Unity | 0 referencias
20     private void OnTriggerEnter2D(Collider2D collision)
21     {
22         if (collision.CompareTag("bala"))
23         {
24             contadorEnemigos.IncrementarEnemigo();
25             Destroy(collision.gameObject);
26         }
27         else if (collision.CompareTag("Player"))
28         {
29             contadorEnemigos.DescontarVidas();
30         }
31         else
32         {
33             return;
34         }
35         Destroy(gameObject);
36     }
37 }

```

Ilustración 53 Código del Archivo Eliminar a Distancia

```
8 public class generales : MonoBehaviour
9 {
10     public string SiguienteEscena = "";
11     // Start is called before the first frame update
12     // Mensaje de Unity | 0 referencias
13     void Start()
14     {
15     }
16
17     // Update is called once per frame
18     // Mensaje de Unity | 0 referencias
19     void Update()
20     {
21     }
22     // 0 referencias
23     public void PasarEscena()
24     {
25         try
26         {
27             SceneManager.LoadScene(SiguienteEscena);
28             //SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex + 1); otro metodo para cambiar escena
29         }
30         catch
31         {
32             print("Error al cambiar de escena :(");
33         }
34     }
35 }
```

Ilustración 54 Código del Archivo Generales